University of Glasgow

School of Computing Science

Blockchain-Based Electronic Voting

Mathematical Modelling and Experimental Evaluation of Bribery, Sybil and Ballot-Stuffing Attacks

A game-theoretic and probabilistic security evaluation,
with exact binomial analysis, Chernoff bounds and
critical-threshold identification

MSc Computing Science

Dissertation

Author, supervisor and submission date to be completed by the candidate.

All results in this document are generated from the accompanying simulation code (seed 42, 2,500 paired trials per vector) and are reproducible bit-for-bit. No real election system, voter record or personal datum is used anywhere in this work.

Abstract

Electronic voting systems built on blockchain ledgers are frequently claimed to be tamper-proof, but immutability of a recorded ballot is a different property from resistance to the manipulation that occurs *before* a ballot is recorded. This dissertation asks a narrower and more answerable question: given a rational attacker with a finite budget, how likely is manipulation to change an election outcome, and what level of countermeasure is actually required to stop it?

Three manipulation vectors are modelled probabilistically and game-theoretically — bribery of genuine voters, Sybil registration of forged identities, and ballot stuffing. Each is expressed as a sequence of independent Bernoulli attempts whose outcome count follows a binomial distribution, with attack success defined as reaching a manipulation threshold m: exactly the number of fraudulent votes needed to overturn the election. Success probabilities are computed three ways — exactly by the binomial survival function, empirically by Monte Carlo simulation, and analytically by a multiplicative Chernoff bound — and the three are cross-validated against one another.

The evaluation runs 2,500 paired randomised trials per vector, comparing a baseline configuration against a protected configuration in which a blockchain-backed audit ledger is combined with digital signatures, identity verification and ballot validation. It is those validation and detection controls, not the ledger, that raise detection and lower acceptance; the ledger contributes integrity, immutability and auditability, and the experiment is designed to keep the two contributions distinguishable. The same underlying scenario is used for both arms of each pair, so the comparison is a matched design rather than two independent samples. A concrete layer complements the statistics: 300 elections are genuinely mined on a proof-of-work chain and attacked, so that abstract success probabilities are tied to actual winner flips.

The protected configuration reduces mean success probability by 81.9% for bribery, 71.8% for Sybil attacks and 73.7% for ballot stuffing, and reduces real winner-flip rates from 44.0%, 54.7% and 55.0% to 1.3%, 11.3% and 8.3% respectively. Chain integrity was never violated across all trials.

Beyond these aggregate results the dissertation contributes two extensions. First, a set of *critical-threshold solvers* inverts the model: rather than reporting how likely an attack is, they report the parameter value at which it stops being viable. The most consequential of these is the critical detection probability q_d^*, which lies between 0.050 and 0.118 across the 4 of 5 benchmark scenarios where detection is the binding constraint — between four and eleven times weaker than the detection the protected configuration actually delivers, and in the fifth scenario the attack never pays at any detection level. Deterrence, on this model, is far cheaper than prevention. Second, a *coalition* model allows one attacker to split a single budget across all three vectors at once. The exact distribution of the combined attack is obtained by convolution rather than approximation, and the mixed strategy strictly outperforms the best single-vector strategy in 4 of 5 scenarios, raising mean success probability from 0.277 to 0.800. Single-vector security analysis therefore understates the risk, while the modelled countermeasures reduce the coalition attack to the same degree as they reduce each vector alone across the benchmark scenarios examined.

Keywords: electronic voting, blockchain, bribery, Sybil attack, ballot stuffing, game theory, Chernoff bound, binomial analysis, security thresholds.

Contents

Abstract 1

1 Introduction 6

1.1 Motivation . . . 6
1.2 Research questions . . . 6
1.3 Contributions . . . 7
1.4 Scope, and what this work is not . . . 7

2 Literature Review 7

2.1 Scope and method of the review . . . 7
2.2 Electronic voting before blockchain . . . 8
2.3 Blockchain-based voting proposals . . . 9
2.4 The critical response: what happens on deployment . . . 9
2.5 Bribery and vote buying as a formal problem . . . 10
2.6 Sybil attacks and identity . . . 11
2.7 Ballot stuffing and statistical election forensics . . . 11
2.8 Probabilistic and game-theoretic security analysis . . . 11
2.9 Synthesis and research gap . . . 12

3 Threat model and mathematical framework 13

3.1 Threat model . . . 13
3.2 Rational voter behaviour (M1–M2) . . . 13
3.3 Attacker budget constraint (M3) . . . 14
3.4 Outcome distributions and success probability (M4–M6, M9–M10, M12–M13) . . . 14
3.5 Detection and survival (M11, M14) . . . 15
3.6 Chernoff bounds (M7–M8) . . . 15
3.7 The two forms are not interchangeable . . . 15
3.8 Two-sided certificate: closing the coverage gap . . . 16
3.9 Notation . . . 16

4 Critical thresholds and the rational attacker 17

4.1 Why invert the model . . . 17

4.2 Voter-side thresholds . . . 17
4.3 Effort and budget thresholds . . . 17
4.4 Margin thresholds . . . 18
4.5 Unified rational attacker across vectors . . . 18
4.6 Coalition attack . . . 19

5 Implementation 19

5.1 Architecture . . . 19
5.2 Numerical care . . . 20
5.3 Testing . . . 20

6 Research Design and Methodology 20

6.1 Research strategy: why simulation . . . 20
6.2 A two-layer architecture, and why both layers are necessary . . . 21
6.3 Variables and operationalisation . . . 22
6.4 What the ledger contributes, and what it does not . . . 22
6.5 Parameter selection and its justification . . . 23
6.6 Paired randomised design . . . 24
6.7 Triangulation of three computational methods . . . 24
6.8 Threshold solver design . . . 25
6.9 Coalition search design . . . 25
6.10 Validation and verification . . . 26
6.11 Figure geometry on a discrete support . . . 26
6.12 Reproducibility protocol . . . 27
6.13 Ethical considerations . . . 27

7 The Experiment in Depth 27

7.1 The five experiments . . . 29
7.2 Anatomy of a single trial . . . 29
7.3 Experiment 1: paired randomised trials . . . 31
7.4 Experiment 2: concrete mined elections . . . 31
7.5 Experiment 3: benchmark scenarios . . . 32
7.6 Experiment 4: threshold solving . . . 32
7.7 Experiment 5: coalition and sensitivity . . . 33

8 Findings **33**

8.1 Overview . . . 33

8.2 F1 — Budget and detection are not symmetric levers . . . 35

8.3 F2 — The three methods agree, and each has a distinct use . . . 37

8.4 F3 — Countermeasures work, but not uniformly . . . 39

8.5 F4 — Chain integrity and election correctness are independent . . . 40

8.6 F5 — Deterrence is far cheaper than prevention . . . 41

8.6.1 Break-even budget . . . 42

8.6.2 The contested band of margins . . . 42

8.6.3 The rational attacker does not spend everything . . . 43

8.7 F6 — Combining vectors defeats what no single vector can . . . 44

8.7.1 No single vector suffices . . . 44

8.7.2 The mixed strategy . . . 46

8.7.3 Robustness of the coalition finding . . . 47

8.7.4 Countermeasures against the coalition . . . 47

8.8 Scale effects . . . 47

9 Discussion **48**

9.1 What the ledger does and does not buy . . . 48

9.2 Deterrence is cheaper than prevention . . . 48

9.3 Single-vector analysis is not conservative . . . 49

9.4 Answers to the research questions . . . 49

10 Limitations and threats to validity **50**

11 Conclusion and future work **50**

References **52**

A Figure catalogue **54**

B Reproducing these results **54**

List of Figures

1 Exact, Monte Carlo and Chernoff comparison 36
2 Voter utility surface and decision threshold 37
3 Two-sided Chernoff certificate 38
4 Three-attack comparison 39
5 Election outcome flip rate 40
6 Critical detection threshold 41
7 Binomial distribution with margin thresholds 43
8 Optimal attacker strategy 44
9 Vector capacity against the threshold 45
10 Coalition against best single vector 46

List of Tables

1 What the literature supplies, and what it leaves open. 12
2 Operationalisation of the model's variables. 22
3 Randomised parameter ranges. Every trial draws one value per symbol uniformly from these ranges; the protected configuration modifies only the acceptance and detection probabilities, never N, m, B, R, P, S or A. 28
4 The five experiments, what each isolates, and what it produces. 28
5 Summary of findings. 34
6 Headline experimental results across 2,500 paired randomised trials per vector. 34
7 Critical thresholds for the five benchmark scenarios under the baseline configuration. . 41
8 Capacity of each vector at full effort, as a multiple of the manipulation threshold m (baseline configuration). 45
9 Optimal multi-vector budget allocation per scenario (baseline configuration). 46

1 Introduction

1.1 Motivation

The appeal of a blockchain-backed ballot box is easy to state: once a vote is written to an append-only, hash-linked ledger and replicated across independent nodes, altering it after the fact is computationally infeasible and, more importantly, immediately visible. That property is real, and the implementation accompanying this dissertation demonstrates it directly — a vote modified after mining is caught by chain validation, with the exact offending block index reported.

The property is also narrower than it is usually made to sound. Immutability protects a vote *after* it is recorded. It says nothing about whether the vote should have existed at all. A ballot cast by a voter who was paid to vote a particular way is recorded just as faithfully as an honest one; so is a ballot cast by a forged identity that passed registration, and so is a ballot injected by an attacker with system access. In each case the ledger is perfectly consistent and the election is still wrong. The security question that matters, then, is not whether the record can be altered, but how much manipulation an attacker can push through the front door, and what it costs them.

That reframing makes the problem quantitative. An attacker has a budget, faces a per attempt cost, and succeeds only if enough fraudulent votes survive the system's defences to overturn the margin. Each of those quantities is a number, and the relationship between them is a probability distribution rather than a binary vulnerability. This dissertation takes that view seriously and works it through.

1.2 Research questions

RQ1. How does the probability that an attack overturns an election outcome depend on the attacker's budget, the per-attempt cost, and the system's detection capability?

RQ2. How accurately do a Monte Carlo estimate and a Chernoff upper bound reproduce the exact binomial probability, and where does each remain useful at scale?

RQ3. How much does the protected configuration — a blockchain-backed audit ledger *combined with* digital signatures, identity verification and ballot validation — reduce attack success, measured as a matched comparison on identical scenarios?

RQ4. What is the *minimum* countermeasure strength that makes each attack non-viable, as opposed to merely less likely to succeed?

RQ5. Does an attacker who combines all three vectors against a single budget outperform one who commits to the best single vector, and does that change the security conclusion?

RQ1–RQ3 are the descriptive questions the base model answers. RQ4 and RQ5 are this dissertation's extensions and are, in the author's view, the more useful pair: a defender cannot act on a probability curve, but can act on a threshold.

1.3 Contributions

1. A complete probabilistic model of three e-voting manipulation vectors (Section 3), implemented with exact binomial computation, Monte Carlo simulation and correctly-signed Chernoff bounds in both tails, with each equation traceable to its implementing function.
2. A matched experimental evaluation over 2,500 paired randomised trials per vector (Section 8), reporting bootstrap confidence intervals rather than point estimates alone.
3. A concrete validation layer in which 300 elections are actually mined on a proof-of-work chain and attacked, tying every abstract probability to a real winner flip (Section 8.5).
4. Seven *critical-threshold solvers* (Section 4) that invert the model and return the parameter value at which an attack ceases to be viable, including the break-even attacker budget and the critical detection probability.
5. A *coalition attack* model (Section 8.7) in which one budget is split across all three vectors, with the combined distribution evaluated exactly by convolution, plus a robustness sweep over 108 cost and capacity configurations.

1.4 Scope, and what this work is not

This is a simulation study with synthetic data. It models no real election, no real voter, and no real deployed system; the candidates are labelled *A* and *B* and have no referent. The blockchain component is a working but educational proof-of-work implementation, not a distributed consensus network with real nodes, network partitions or economic security. Countermeasures are modelled as probability parameters rather than as specific cryptographic constructions: the model says what happens if identity verification catches a forged registration with probability p_d, not how such verification is built.

The work is also deliberately confined to manipulation of vote *inputs*. Attacks on the consensus layer itself — majority hash-power attacks, chain reorganisation, eclipse attacks — are out of scope, as are coercion and receipt-freeness, which raise questions of ballot secrecy rather than of ballot count. Section 10 returns to these boundaries and to what would be needed to cross them.

2 Literature Review

2.1 Scope and method of the review

This review covers four bodies of work that the dissertation draws together, and it is organised by the question each body answers rather than chronologically.

1. **What has been built, and what is claimed for it?** Electronic and blockchain-based voting systems, from cryptographic verifiable-voting schemes to ledger-backed proposals (Sections 2.2–2.3).
2. **What breaks when it is deployed?** The empirical security literature: analyses of fielded internet-voting systems and the critical response to blockchain voting specifically (Section 2.4).

3. **How is manipulation modelled formally?** Bribery as a computational problem, coercion and receipt-freeness, Sybil identity attacks, and the statistical forensics of fraudulent tallies (Sections 2.5–2.7).
4. **What tools exist for reasoning about a rational, resource-bounded attacker?** Concentration inequalities and game-theoretic security analysis (Section 2.8).

Sources were located by targeted search of the primary security venues (USENIX Security, ACM CCS, Financial Cryptography, IEEE S&P and their workshops), the artificial-intelligence and economics literature for the manipulation-theory strand, and two recent systematic reviews of blockchain voting [16, 18] used as entry points into the applied literature. Priority was given to peer-reviewed primary sources; where a widely-cited result exists only as a technical report or research blog post, as with the Dark DAO construction [6], this is stated explicitly rather than disguised by citation formatting.

Section 2.9 closes the review by identifying what the four bodies of work leave unaddressed, which is the gap this dissertation occupies.

2.2 Electronic voting before blockchain

The security requirements for an electronic election were substantially settled before distributed ledgers entered the conversation, and it is worth restating them because blockchain proposals are frequently evaluated against a narrower list.

Verifiability. A voter should be able to confirm that their ballot was recorded as cast, and any observer should be able to confirm that the recorded ballots were tallied correctly. Adida's Helios [1] is the canonical open-audit implementation: it publishes encrypted ballots to a public bulletin board and proves the tally correct with homomorphic encryption and zero-knowledge proofs, so that correctness rests on mathematics rather than on trusting the election authority. Helios matters to the present work as a counterexample to a common framing: it delivers public verifiability with no ledger, no chain and no consensus mechanism, which shows that the verifiability argument for blockchain voting is not an argument for blockchain specifically.

Ballot secrecy and its stronger forms. Secrecy alone is insufficient if a voter can *prove* how they voted, because a provable vote is a sellable vote. Benaloh and Tuinstra [3] formalised *receipt-freeness*: the requirement that a voter cannot construct a convincing proof of their choice even if they want to. Juels, Catalano and Jakobsson [10] strengthened this to *coercion-resistance*, covering an adversary who can interact with the voter during voting, demand credentials, or require abstention; their scheme lets a coerced voter surrender a fake credential indistinguishable from a real one.

This distinction sets the boundary of the present work. Bribery, as modelled here, presupposes that the briber can verify compliance well enough for the transaction to be worth making. A system that is genuinely receipt-free undermines that presupposition at the root, which is a different and stronger defence than the detection-based one modelled in this dissertation. Section 10 returns to this.

2.3 Blockchain-based voting proposals

Nakamoto's chained proof-of-work construction [15] supplied a mechanism with obvious surface appeal for elections: an append-only, hash-linked, replicated log in which retrospective edits invalidate every subsequent hash and are therefore detectable by anyone holding a copy.

The most rigorous academic instantiation is McCorry, Shahandashti and Hao's Open Vote Network [13], a boardroom-voting protocol implemented as an Ethereum smart contract, which achieves maximum voter privacy in the sense that a vote can only be revealed by full collusion of all remaining participants, and which is self-tallying: no trusted authority computes the result. It is a genuine cryptographic contribution rather than a ledger with ballots written into it. Its scope, however, is explicitly the boardroom — tens of voters, all known to one another — and its cost and collusion properties do not transfer to a public election.

The wider applied literature is large and, by the assessment of its own systematic reviewers, uneven. Ohize et al. [16] survey architectures and trends across the field and identify scalability, post-quantum security, voter authentication, regulatory frameworks and cross-chain interoperability as the persistent open problems. Rahul, Gulia and Gill [18] review 55 journal articles and reach a similar conclusion, noting that security, transparency and trust remain unresolved worldwide rather than settled by any existing proposal. Two observations from these reviews shape the present work. First, *voter authentication* is repeatedly named as an open problem, which is precisely the Sybil vector modelled here. Second, the surveyed proposals are overwhelmingly evaluated on functionality, throughput and cryptographic properties rather than on quantified resistance to a budgeted attacker — the gap this dissertation addresses.

2.4 The critical response: what happens on deployment

Against the proposal literature stands a smaller and considerably more sobering body of empirical analysis.

Springall et al. [20] examined Estonia's I-voting system, the most established national internet-voting deployment, by observing operations directly and reconstructing the system in the laboratory. They reported serious procedural lapses and architectural weaknesses sufficient, in their assessment, for a state-level attacker to alter results undetectably, and recommended discontinuing the system. The finding relevant here is not the specific vulnerabilities but the location of them: the client platform and the operational procedures surrounding the count, not the cryptography.

Specter, Koppel and Weitzner [19] reverse-engineered Voatz, the first internet-voting application used in U.S. federal elections and marketed on its use of a blockchain. They found flaws allowing an adversary to alter, stop or expose a voter's ballot, a side-channel through which a passive network observer could recover the secret ballot, and a dependence on third-party services with attendant privacy exposure; West Virginia and a Washington county withdrew the application from 2020 primaries following the analysis. The architectural point is the important one: the blockchain sat behind an application whose compromise made the ledger irrelevant, because votes were corrupted before they reached it.

Park, Specter, Narula and Rivest [17] generalise these observations into a direct argument that blockchain

voting is worse than internet voting rather than better. Their case is that blockchains address a problem elections do not have — disagreement about which transactions occurred among mutually distrusting parties — while introducing new failure modes, and that the genuinely hard problems of elections, namely software independence, secret ballots, voter authentication and client-side integrity, are untouched. They further note that a blockchain vote is a *transparent* record, which is in tension with ballot secrecy rather than supportive of it.

This dissertation adopts that critique as a premise to be tested quantitatively rather than as a conclusion to be restated. The experimental design deliberately grants the ledger everything its proponents claim — integrity is verified continuously and, as Section 8 reports, was never violated in any trial — and then measures how many elections are stolen anyway.

2.5 Bribery and vote buying as a formal problem

Three distinct traditions model vote buying, and this dissertation borrows from all three.

Computational. Faliszewski, Hemaspaandra and Hemaspaandra [9] formalise bribery as a decision problem: given per-voter prices and a budget, can the briber make a preferred candidate win? They classify the complexity across voting rules, establishing that some systems make optimal bribery NP-hard — complexity as a defensive barrier. The framing is worst-case, combinatorial and deterministic: a bribed voter votes as paid.

Economic. Becker's analysis of crime as a decision under risk [2] models an offender weighing gain against the probability of detection multiplied by the penalty, and derives the policy conclusion that deterrence can be achieved either by raising detection or by raising penalties. Equation (2) in this dissertation is exactly that calculation applied to a bribed voter, and the critical-detection threshold of Section 4 is the policy question in Becker's sense: how much detection is enough?

Blockchain-specific. Daian, Kell, Bentov and Juels [6] describe on-chain vote buying and the *Dark DAO*: a smart contract that pays for votes and enforces the payment trustlessly, using trusted execution to verify compliance without revealing the buyer. Their argument deserves emphasis in the present context because it inverts the usual claim. A blockchain does not merely fail to prevent vote buying; by making votes cryptographically attributable and payments programmable and escrowed, it *improves* the market for votes, removing the counterparty risk that limits bribery in traditional settings. This is a technical-report and research-blog contribution rather than a peer-reviewed paper, and is cited as such, but it is widely referenced and directly motivates treating bribery, not tampering, as the primary threat to a ledger-based election.

The gap across all three: the computational tradition treats bribery deterministically, the economic tradition treats the individual decision without an electorate-level outcome model, and the blockchain-specific work is a mechanism proposal rather than a quantified risk model. None produces the quantity a defender needs — the probability that a given budget overturns a given margin against a given detection capability.

2.6 Sybil attacks and identity

Douceur [7] established that, absent a trusted certification authority, a single entity in a distributed system can present arbitrarily many distinct identities, and that no purely local resource test reliably distinguishes them. For an election the consequence is immediate and fatal: one-person-one-vote is the entire premise, and a Sybil attack attacks the premise rather than the mechanism.

Responses divide into those that reintroduce a trusted authority and those that attempt to establish uniqueness without one. Borge et al. [4] propose proof-of-personhood, binding digital identities to physical presence at pseudonym parties, explicitly to recover one-per-person semantics in permissionless systems. The approach illustrates both the necessity and the difficulty: physical attendance is a strong Sybil barrier and an equally strong barrier to accessibility, which is what electronic voting exists to improve.

This dissertation does not model any specific identity mechanism. It parameterises the outcome instead — a forged registration passes with probability q_s and is caught with probability p_d — which is weaker than assuming a trusted authority and is deliberately conservative. It also produces the finding, reported in Section 8, that identity verification rather than ledger integrity is where the residual risk concentrates, consistent with voter authentication being repeatedly flagged as an open problem in the systematic reviews [16, 18].

2.7 Ballot stuffing and statistical election forensics

Ballot stuffing — injecting fraudulent ballots not corresponding to any voter — is the least cryptographically interesting of the three vectors and, historically, among the most common. The relevant literature is statistical rather than cryptographic.

Klimek, Yegorov, Hanel and Thurner [11] show that systematic irregularities leave detectable signatures in the joint distribution of turnout and vote share across polling units: incremental stuffing and extreme fraud produce characteristic distortions absent from clean elections, and the authors identify these fingerprints in several national elections. The methodological lesson carried into this dissertation is that fraud is bounded by *plausibility* as much as by opportunity. An attacker who injects ballots without limit produces a turnout figure that requires no probabilistic detection to notice. This is the justification for the capacity ceilings imposed in the coalition model (Section 4.6), which are otherwise a modelling choice without grounding: they stand in for the anomaly detection that election forensics performs.

2.8 Probabilistic and game-theoretic security analysis

The final strand supplies the analytical machinery.

Concentration inequalities. Chernoff's bound [5], in the multiplicative form given by Mitzenmacher and Upfal [14], bounds the probability that a sum of independent Bernoulli variables deviates from its mean, and does so exponentially in the deviation. For this dissertation its value is specific: it converts an empirical statement into a provable one. A margin certified safe by the bound is safe irrespective of how many simulation runs were performed, which is what makes the certified margin of Section 4 a security

claim rather than an observation.

Rational attackers. Eyal and Sirer [8] demonstrated with selfish mining that a blockchain protocol assumed secure under an honest-majority model can be broken by a rational minority that deviates profitably — security depending on incentives, not only on cryptography or on hash-power share. Liu et al. [12] survey the resulting body of game-theoretic blockchain analysis, covering mining strategy, security games and incentive design.

Two points transfer directly. First, the relevant question about an attacker is what they would *choose* to do, not what they are capable of doing: Section 8.6 shows that a budget-exhausting attacker performs strictly worse than a rational one, so a defence tuned against maximum expenditure is tuned against the wrong opponent. Second, deterrence is a legitimate security goal distinct from prevention — making an attack unprofitable is cheaper than making it impossible, and the gap between those two thresholds is quantified in this dissertation.

2.9 Synthesis and research gap

Table 1 summarises what each strand supplies and what it leaves open.

Table 1. What the literature supplies, and what it leaves open.

Strand	Supplies	Leaves open
Blockchain voting proposals [13, 16, 18]	Architectures, cryptographic protocols, throughput evaluation	No quantified success probability against a budgeted attacker
Empirical critiques [20, 19, 17]	Evidence that deployments fail upstream of the ledger	Qualitative argument; no model of how much manipulation gets through
Bribery theory [9, 2, 6]	Complexity classification; the rational-offender calculus; on-chain vote-buying mechanisms	Deterministic or individual-level; no electorate-scale outcome distribution
Sybil and identity [7, 4]	Impossibility without trusted certification; countermeasure designs	No probabilistic model of partial identity verification
Election forensics [11]	Statistical signatures of stuffing; plausibility as a real constraint	Detection after the fact, not a design threshold beforehand
Probabilistic and game-theoretic tools [5, 14, 8, 12]	Concentration bounds; rational-deviation analysis	Not yet applied to e-voting manipulation as an integrated model

Three gaps follow, and they define the contribution of this dissertation.

Gap 1: no quantified attacker model for e-voting countermeasures. The proposal literature evaluates systems on cryptographic properties and performance; the critical literature establishes qualitatively that deployments fail. Neither produces the probability that a specific budget overturns a specific margin against a specific detection capability. Sections 3 and 8 supply exactly that, over 2,500 matched randomised trials per vector.

Gap 2: probabilities are reported where thresholds are needed. A defender cannot act on a success-probability curve. The inverse question — what detection probability, what budget ceiling, what margin makes this attack non-viable — is not asked in the e-voting literature, although Becker's economics

of deterrence [2] and the game-theoretic blockchain work [8, 12] both imply it should be. Section 4 formulates seven such thresholds and solves them.

Gap 3: attack vectors are evaluated in isolation. Every source reviewed treats bribery, Sybil attacks and ballot stuffing separately. No reviewed work models an attacker allocating one budget across several vectors simultaneously. Section 8.7 shows this omission is not conservative: two of the three vectors are individually infeasible in every benchmark scenario, yet their combination succeeds decisively in four of five.

3 Threat model and mathematical framework

3.1 Threat model

A single attacker attempts to change the outcome of a two-candidate election with N enrolled voters. Let the honest tally leave the attacker's preferred candidate behind by some margin; the *manipulation threshold* m is the number of additional fraudulent votes needed to overturn that margin, so that

$$m = (\text{opponent votes} - \text{target votes}) + 1, \tag{1}$$

and the attack succeeds precisely when at least m fraudulent votes survive every countermeasure and reach the final count. Throughout this dissertation, "attack success" means exactly this event and nothing weaker.

Three vectors are available.

Bribery.
The attacker offers a reward R to genuine voters, constrained by a total budget B. A targeted voter accepts only if doing so is rationally incentivised. Each bribed vote is detected, and therefore discarded, with probability q.

Sybil registration.
The attacker submits S forged identities. Each passes registration with probability q_s and is independently detected afterwards with probability p_d.

Ballot stuffing.
The attacker injects A fraudulent ballots. Each is accepted with probability q_b and independently detected with probability p_b.

Attempts within a vector are independent and identically distributed. This is a modelling assumption with real consequences, and Section 10 examines what correlated behaviour would do to the results.

3.2 Rational voter behaviour (M1–M2)

A voter offered a bribe faces a decision under risk. Accepting yields R if undetected and costs a penalty P if detected, so expected utility is

$$U_v = (1-q)R - qP. \tag{2}$$

A risk-neutral voter accepts when $U_v > 0$, which rearranges to the incentive-compatibility condition

$$R > \frac{qP}{1-q}, \qquad q \neq 1. \tag{3}$$

At $q = 1$ the right-hand side is undefined; the implementation returns no threshold rather than substituting a value, and falls back on Equation (2), which remains well-defined and gives $U_v = -P$.

Equation (2) carries more weight in this model than its simplicity suggests. It is a *gate*: if bribery is not incentive-compatible, no bribe is accepted, and the attack contributes zero fraudulent votes no matter how large the budget. This is why bribery behaves discontinuously in the results while the other two vectors degrade smoothly.

3.3 Attacker budget constraint (M3)

With reward R per voter and budget B, the attacker can approach at most

$$T = \min\left(N, \left\lfloor \frac{B}{R} \right\rfloor\right) \tag{4}$$

voters — limited either by money or by the size of the electorate. The cases $B = 0$, $R = 0$ and $B < R$ all yield $T = 0$ explicitly rather than an undefined division.

3.4 Outcome distributions and success probability (M4–M6, M9–M10, M12–M13)

Because attempts are independent Bernoulli trials, each vector's surviving-fraud count is binomial:

$$X \sim \text{Binomial}(T, 1-q) \qquad \text{bribery, surviving bribed votes} \tag{5}$$

$$Y \sim \text{Binomial}(S, q_s) \qquad \text{Sybil, accepted identities} \tag{6}$$

$$Z \sim \text{Binomial}(A, q_b) \qquad \text{ballot stuffing, accepted ballots} \tag{7}$$

with attack success probability given in every case by the survival function

$$\mathbb{P}(X \geq m) = \sum_{k=m}^{n} \binom{n}{k} p^k (1-p)^{n-k}, \qquad \mathbb{E}[X] = \mu = np. \tag{8}$$

This is computed exactly, via the regularised incomplete beta function underlying `scipy.stats.binom.sf`, and never by normal or Poisson approximation.

3.5 Detection and survival (M11, M14)

Detection applies independently to each fraudulent item, so the probability that all n escape, and its complement, are

$$\mathbb{P}(\text{all escape}) = (1-p)^n, \qquad \mathbb{P}(\text{at least one caught}) = 1-(1-p)^n. \tag{9}$$

Both are evaluated in log space, via `log1p` and `expm1`, so that they remain accurate for the small p and large n combination where a naive implementation loses all significance.

A forged identity or injected ballot must clear *two* independent gates to count: acceptance and non-detection. Under independence the surviving count is therefore

$$W \sim \text{Binomial}\big(n,\ q_{\text{accept}}(1-p_{\text{detect}})\big), \tag{10}$$

and it is W, not the raw acceptance count, that is compared against the threshold m throughout the results. The independence of the two gates is an assumption, flagged as such; a system whose detector preferentially re-examines items that only barely passed registration would violate it.

3.6 Chernoff bounds (M7–M8)

For X a sum of independent Bernoulli variables with mean μ, the multiplicative Chernoff bounds are

$$\mathbb{P}\big(X \geq (1+\delta)\mu\big) \leq \exp\Big(\mu\big[\delta-(1+\delta)\ln(1+\delta)\big]\Big), \qquad \delta > 0 \tag{11}$$

$$\mathbb{P}\big(X \leq (1-\delta)\mu\big) \leq \exp\left(\frac{-\mu\delta^2}{2}\right), \qquad 0 < \delta < 1. \tag{12}$$

Two implementation points deserve emphasis, because both are easy to get wrong and both are guarded by tests.

The sign in Equation (12) is negative and is hard-coded as such. A positive exponent produces a "bound" exceeding 1, which is vacuous as a probability statement while still looking plausible in output; a dedicated regression test asserts that the returned value never exceeds the correctly-signed expression and is strictly below the incorrectly-signed one.

The bound is applied only where it is mathematically valid. Given (n, p, m) the implementation first determines whether m lies in the upper tail ($m > \mu$), the lower tail ($m < \mu$), or neither, and returns *no bound*, with a machine-readable reason, outside those regions.

3.7 The two forms are not interchangeable

Because it was raised in review, it is worth stating explicitly that the $(1+\delta)$ factor in Equation (11) cannot be replaced by $(1-\delta)$. The factor is not a stylistic choice: it records which side of the mean the

threshold lies on. Substituting one for the other does not weaken or strengthen the bound, it falsifies it.

Evaluating the (incorrect) claim $\mathbb{P}(X \geq (1-\delta)\mu) \leq [e^{-\delta}/(1-\delta)^{(1-\delta)}]^{\mu}$ for $\delta \in \{0.05, 0.10, 0.20\}$ across four (n, p) settings — twelve cases — the inequality fails in eleven of them, and fails by wide margins: at $n = 2000$, $p = 0.45$, $\delta = 0.20$ the true probability is 1.0000 against a claimed bound of 4.0×10^{-9}. The reason is structural. The threshold $(1-\delta)\mu$ lies *below* the mean, so $\mathbb{P}(X \geq (1-\delta)\mu)$ approaches one, while the expression decays exponentially in μ. The correctly-signed upper-tail form bounds every one of the same cases, and the $(1-\delta)$ form is valid on the tail it belongs to, $\mathbb{P}(X \leq (1-\delta)\mu)$. All three statements are regression-tested rather than merely asserted here.

3.8 Two-sided certificate: closing the coverage gap

The review point did, however, expose a real limitation. The dispatcher returns a bound only when $m > \mu$, which left every scenario with $m \leq \mu$ — 20.6% to 30.6% of trials depending on the vector — with no analytic statement at all. Those are not marginal cases: $m \leq \mu$ is precisely the regime in which the attack is *likely to succeed*, and therefore the more alarming half of the parameter space.

The remedy is to bound whichever tail the threshold actually falls in. Where $m > \mu$, the upper-tail form certifies the success probability from above. Where $m \leq \mu$, the failure event $X \leq m-1$ is itself a lower-tail event, so the $(1-\delta)$ form of Equation (12) bounds $\mathbb{P}(X \leq m-1)$ from above and thereby certifies success from *below*:

$$\mathbb{P}(X \geq m) \;\geq\; 1 - \exp\left(\frac{-\mu\delta^2}{2}\right), \qquad \delta = 1 - \frac{m-1}{\mu}. \tag{13}$$

Both are one-sided guarantees, and which one is available is a property of the scenario rather than a choice. The security readings differ usefully: an upper bound says the attack cannot succeed with probability greater than B, while a lower bound says it succeeds with probability at least $1-B$ — the stronger claim to be able to make about an adversary.

With the certificate in place every scenario carries a statement. Across all 15,000 scenario evaluations the certified interval contains the exact binomial value in 100% of cases, with a mean interval width of 0.018 for bribery, 0.044 for Sybil and 0.043 for ballot stuffing, and no scenario left uncovered. Figure 3 validates both directions against the exact value.

3.9 Notation

This work uses q for the *detection* probability throughout, so that a bribed vote survives with probability $1-q$. Some presentations of the same model write q for the success probability directly. The two are related by $q_{\text{alt}} = 1-q$ and nothing in the mathematics changes; the detection convention is kept here because it composes directly with Equation (2), where q reads naturally as "risk of being caught".

4 Critical thresholds and the rational attacker

4.1 Why invert the model

Sections above answer "how likely is this attack to succeed?". A defender designing a system needs the inverse: "what would have to be true for this attack to stop being worth mounting?" The answer to the first question is a curve; the answer to the second is a number, and a number can be turned into a requirement. The solvers below produce those numbers. Each returns *no value*, with a reason, when the threshold does not exist in the searched domain — a threshold that does not exist is reported as absent rather than as an endpoint.

4.2 Voter-side thresholds

The same indifference condition $U_v = 0$ can be solved for either of its two free variables, giving a pair of thresholds that answer the attacker's question and the defender's question respectively. Solving for the reward gives the smallest bribe a rational voter accepts at a given detection probability:

$$R^* = \frac{qP}{1-q}, \tag{14}$$

undefined at $q = 1$, where no finite reward is incentive-compatible while $P > 0$. This is the attacker's per-voter price, and it is what makes the budget constraint $T = \min(N, \lfloor B/R \rfloor)$ bite.

Solving the same condition for q instead gives the detection probability at which the targeted voter is exactly indifferent:

$$q_v^* = \frac{R}{R+P}. \tag{15}$$

This is the single most actionable quantity in the bribery model. It depends only on the ratio of reward to penalty — not on the electorate size, not on the margin, not on the attacker's budget. Above q_v^* a rational voter refuses, the attack collapses to $T = 0$, and no budget rescues it. Figure 2 shows the surface and the threshold together.

4.3 Effort and budget thresholds

The minimum effort needed for even odds of reaching the threshold,

$$T_{\min} = \min\{\, n : \mathbb{P}(X \geq m) \geq \alpha \,\}, \tag{16}$$

is found by binary search, which is exact because $\mathbb{P}(X \geq m)$ is non-decreasing in n for fixed m and p.

The economic thresholds require care about what "the attacker's utility" means. The attacker's expected utility from T bribery attempts is

$$U(T) = \mathbb{P}(X \geq m)V \; - \; TR \; - \; \big[1-(1-q)^T\big]D, \tag{17}$$

where V is the payoff from flipping the outcome and D the penalty incurred if the attack is noticed at all. A rational attacker may always decline, so $\max_{T\geq 0} U(T) \geq 0$ identically — a quantity floored at zero cannot be solved for a zero crossing. The thresholds are therefore defined on the *signed* payoff of actually attacking, $\max_{T\geq 1} U(T)$, which is negative precisely when attacking is a losing proposition. With that definition:

$$B^* = \min\{\, B : \max_{T\geq 1} U(T) > 0 \,\} \qquad \text{break-even budget} \tag{18}$$

$$q_d^* = \min\{\, q \in [0,1] : \max_{T\geq 1} U(T) \leq 0 \,\} \qquad \text{critical detection probability} \tag{19}$$

Both are located by bisection on a monotone function: the feasible set grows with B, while rising q lowers the survival probability and raises the expected penalty simultaneously. By construction $q_d^* \leq q_v^*$, and the gap between them is the region in which deterrence depends on the attacker being rational rather than on the attack being impossible.

4.4 Margin thresholds

Two thresholds bracket the contested region of margins. The largest margin the attacker overturns at least at probability α,

$$m_\alpha = \max\{\, m : \mathbb{P}(X \geq m) \geq \alpha \,\}, \tag{20}$$

and the smallest margin the Chernoff bound certifies as safe at tolerance ε,

$$m_\varepsilon = \min\{\, m : \exp\big(\mu[\delta - (1+\delta)\ln(1+\delta)]\big) \leq \varepsilon \,\}, \qquad \delta = \tfrac{m}{\mu} - 1. \tag{21}$$

The distinction between them is the distinction between an empirical and a provable statement. m_α says what the attacker usually manages; m_ε says what the attacker cannot manage except with probability at most ε, regardless of how the simulation happened to run. Margins between the two are precisely where the exact binomial calculation, rather than either heuristic, decides the outcome. Figure 7 draws all three thresholds on a single distribution.

4.5 Unified rational attacker across vectors

The three vectors share a structure — independent attempts, a per-attempt cost, a survival probability, a detection probability, a capacity ceiling — and can therefore be optimised through one interface. Writing c for the unit cost and p_{surv} for the survival probability of a single attempt, the utility of n attempts is

$$U(n) = \mathbb{P}(W \geq m)V \; - \; nc \; - \; \left[1-(1-p_{\text{detect}})^n\right]D, \qquad W \sim \text{Binomial}(n, p_{\text{surv}}), \tag{22}$$

which specialises to Equation (17) for bribery, with the additional rational-voter gate of Equation (2) applied first. Sybil and ballot stuffing carry no such gate: there is no third party whose consent the attacker needs.

4.6 Coalition attack

Nothing forces an attacker to choose one vector. Splitting a single budget B in shares (w_1, w_2, w_3) across the three gives a total surviving-fraud count

$$W_{\text{total}} = W_{\text{bribery}} + W_{\text{Sybil}} + W_{\text{ballot}}, \tag{23}$$

a sum of independent binomials with *different* success probabilities — which is not itself binomial, and has no closed form. It is evaluated here exactly, by convolving the three probability mass functions. The evaluation exploits one useful identity: since $\mathbb{P}(W \geq m) = 1 - \mathbb{P}(W \leq m-1)$, and no combination summing to at most $m-1$ can involve a component larger than $m-1$, truncating every component distribution at $m-1$ before convolving is exact rather than approximate. That keeps the transform length at m rather than at the total attempt count, which is what makes national-scale margins tractable.

The Chernoff bound applies to W_{total} unchanged, because Equation (11) is stated for sums of independent Bernoulli variables of arbitrary and possibly unequal parameters; only the aggregate mean is needed.

5 Implementation

5.1 Architecture

The system has two complementary layers, and the separation is deliberate.

The **concrete layer** is a working blockchain e-voting application: SHA-256 block hashing, proof-of-work mining with adjustable difficulty, chain validation, and a voting service with voter registration, role-based election administration, duplicate-vote prevention and result publication. It produces actual elections with actual winners, and demonstrates tamper detection directly — a vote edited after mining is caught by chain validation, which reports the offending block index.

The **abstract layer** is the statistical framework: exact binomial computation, Chernoff bounds, Monte Carlo simulation, the threshold solvers and the game-theoretic optimisers. It runs thousands of paired trials in seconds, which would be infeasible against genuinely mined chains.

Each layer answers what the other cannot. The abstract layer provides statistical power; the concrete layer provides ground truth, confirming that its probabilities correspond to real winner flips rather than to an artefact of the abstraction. Where the two are compared (Section 8.5) they agree in ordering and magnitude.

5.2 Numerical care

Several implementation decisions matter for correctness rather than for style, and are recorded here because a reader reproducing this work would otherwise have to rediscover them.

Survival probabilities use log-space evaluation, so $(1-p)^n$ is computed as $\exp(n\ln(1-p))$ via `log1p`, avoiding the catastrophic cancellation that a naive implementation suffers for small p and large n. All randomness flows through a single seeded `numpy.random.Generator`; no global random state is used anywhere, which is what makes the bit-for-bit reproducibility claim in Section 6.12 enforceable by test. Binomial parameters are clamped into valid ranges before use, so floating-point drift that pushes a probability to $1+10^{-17}$ cannot produce a domain error deep inside a library call. The coalition convolution switches to an FFT for long vectors and clips the result at zero, since the true coefficients are probabilities and any negative value is round-off.

5.3 Testing

The suite contains 122 tests. Beyond routine coverage, several encode properties that the mathematics requires and that a plausible-looking implementation can silently violate: that the lower-tail Chernoff bound carries a negative exponent; that the bound is never returned outside its region of validity; that $T=0$ rather than a division error when $B=0$, $R=0$ or $B<R$; that identical seeds produce identical output and different seeds do not; that the exact convolution of the coalition distribution matches a 200,000-draw Monte Carlo simulation of the same three binomials; that a mixed strategy is never worse than the best pure one, since the pure strategies are corners of the same simplex; and that each threshold solver's answer actually brackets the crossing it claims to find.

6 Research Design and Methodology

6.1 Research strategy: why simulation

The research questions ask what *would* happen to an election under attack. That rules out the two obvious empirical strategies. A field experiment is unethical and illegal: it would require attempting to corrupt a real election. Observational study of historical fraud suffers from severe selection bias, because only detected fraud is observable and detected fraud is by definition the fraud that failed at the part this dissertation measures.

The strategy adopted is therefore **computational simulation with analytical validation**: construct a formal model of the attack, compute its consequences exactly where closed forms exist, simulate where they do not, and validate the two against each other. The claims that result are conditional — "under this model, with these parameters, the following follows" — and the burden of the design is to make the conditions explicit, the computation correct, and the results reproducible.

Three commitments follow, and they shape everything below.

Exactness before estimation. Where a quantity has a closed form it is computed exactly, not estimated.

Attack success probability is the binomial survival function (Equation (8)), evaluated through the regularised incomplete beta function; the coalition distribution is an exact convolution (Section 4.6); the threshold solvers are exact binary searches or bisections on monotone functions. Monte Carlo is used to *validate* these exact values and to reach quantities that have no closed form, not as the primary method. This inverts the more common simulation-study design, and it is what allows the Monte Carlo error itself to be reported as a result (Section 8.3).

Falsifiability by construction. Wherever a modelling decision could flatter the conclusion, the design includes a mechanism that would expose it. The Chernoff bound is checked against the exact value in every trial, so a wrongly-implemented bound would show as a violation rather than as a plausible number. The coalition convolution is checked against a 200,000-draw Monte Carlo simulation in the test suite. Reproducibility is enforced by a test that fails if two runs at the same seed differ, and by a second test that fails if two runs at *different* seeds agree — so the first test cannot pass vacuously.

Assumptions declared and swept. Every modelling assumption is recorded in `ASSUMPTIONS.md` and cross-referenced from the module that relies on it. Where an assumption is both consequential and ungrounded — the relative costs of the three vectors, the capacity ceilings — it is not merely declared but swept, and the conclusion is reported as a function of it (Section 7.7).

6.2 A two-layer architecture, and why both layers are necessary

The implementation has two layers that answer the same questions by different means.

The **concrete layer** is a working blockchain e-voting application: SHA-256 block hashing, proof-of-work mining, chain validation, voter registration, role-based election administration, duplicate-vote prevention and result publication. Attacking it produces an actual winner change on an actual mined chain.

The **abstract layer** is the mathematical framework: exact binomial evaluation, Chernoff bounds, Monte Carlo, the threshold solvers and the game-theoretic optimisers. It produces probabilities rather than outcomes.

Neither layer alone would support the conclusions. The abstract layer supplies statistical power — 2,500 paired trials complete in 7.9 seconds, whereas mining that many elections is infeasible — but every one of its numbers is a probability computed under a model, and a model can be wrong in ways that are invisible from inside it. The concrete layer supplies ground truth — a real tally, a real winner, a real chain whose integrity is checked after every attack — but at a sample size (300 elections) too small for the parameter sweeps.

Running both is a complementary analysis, not a validation of one estimand by the other. The concrete experiment uses different electorate and attack-resource ranges and derives its threshold from a realised two-candidate margin, whereas E1 uses the abstract parameter distribution. Section 8.5 therefore compares direction and interpretation only; it does not claim that the two rates estimate the same quantity.

6.3 Variables and operationalisation

Table 2 states how each construct is operationalised. The distinction that matters most is between the three *independent* variables the experiment manipulates and the *control* variables that are held identical across the paired arms.

Table 2. Operationalisation of the model's variables.

Role	Variable	Operationalisation
Independent	System configuration	Two levels: baseline (weak detection and validation) and protected (blockchain-style audit, signatures, identity verification, ballot validation — modelled as higher detection and lower acceptance probabilities).
Independent	Attack vector	Three levels: bribery, Sybil registration, ballot stuffing; plus the coalition of all three in Section 8.7.
Independent	Attacker resources	Budget B and reward R, entering through the feasibility constraint $T = \min(N, \lfloor B/R \rfloor)$.
Dependent	Attack success probability	$\mathbb{P}(W \geq m)$, where W counts fraudulent votes surviving every countermeasure. Computed exactly; also estimated by Monte Carlo and bounded by Chernoff.
Dependent	Election flip	Binary: did the winner of a concretely mined election change? Measured only in the concrete layer.
Dependent	Attacker expected utility	Equation (17): payoff weighted by flip probability, less cost, less expected penalty.
Control	Scenario structure	N, m, B, R, P, S, A — identical between the two arms of every pair by construction.
Control	Random stream	A single seeded generator; identical seed gives identical output bit-for-bit.

6.4 What the ledger contributes, and what it does not

Because the protected configuration bundles several controls together, it is worth stating precisely which component does what before any result is reported. The pipeline is ordered, and the ordering is the whole point:

fraud attempt → **acceptance / identity / signature / ballot validation** → **detection** → surviving votes only → **ledger records them**

Every fraudulent vote that is stopped in this model is stopped by the validation or detection stage. The ledger sits at the end of the pipeline and records whatever survives; it contributes integrity, immutability, auditability and tamper evidence, and it contributes nothing whatever to deciding *whether a vote should have been cast*. A blockchain cannot tell that a genuine voter was paid, that a registration was forged, or that a ballot was injected — those are determined upstream, by mechanisms that would work equally well behind a conventional append-only audit log.

This distinction is preserved in the experimental design rather than merely asserted in the prose. The concrete layer runs the chain's own validity check after every attack, so the two properties are measured

separately: the ledger's integrity, and the election's correctness. The *Findings* chapter reports that these come apart completely — integrity held in every trial, and elections were flipped anyway.

Nothing in this dissertation should therefore be read as showing that "blockchain prevents bribery", or Sybil attacks, or ballot stuffing. The claim supported by the results is narrower and conditional: under the assumptions of this model and over the sampled parameter ranges, a protected architecture combining ledger-backed auditability with identity verification, signature validation, ballot validation and raised detection substantially reduces attack feasibility.

The dependent variable deserves one clarification. "Attack success" is defined throughout as reaching the manipulation threshold m, which is the number of additional fraudulent votes needed to overturn the margin (Equation (1)). It is not defined as "some fraud occurred" or "fraud was undetected". This is a deliberately demanding definition: an attack that places a hundred fraudulent votes into an election it fails to flip counts, here, as a failure.

6.5 Parameter selection and its justification

Parameter ranges (Table 3) were selected to span plausible deployment conditions rather than to be estimated from data, because for most of these quantities no defensible estimate exists. The probability that a forged voter registration is detected is not published by any electoral authority in a form that would support a point estimate. Three consequences follow, and the design accommodates all three.

Ranges, not points. Every parameter is sampled from a range on every trial, so no result depends on a single assumed value. The electorate is drawn from the configured range 200–5,000, realising 200 to 4,999 across the trials actually run, and the threshold m is itself randomised as 2–25% of N, so the reported means are averages over a distribution of scenarios rather than properties of one configuration.

Direction of bias stated. Where a choice could flatter either side it is made against the conclusion being argued. Legitimate ballots always pass validation, so the model never credits the defence with catching fraud by accident. Detection is applied independently per item, which is more favourable to the attacker than correlated detection, since an audit that catches one item usually catches its batch.

Sensitivity where it matters. The coalition analysis depends on four constants with no empirical basis — two relative costs and two capacity ceilings. Rather than defend particular values, the analysis re-runs across 108 combinations of them and reports the conclusion's stability (Section 7.7).

The benchmark scenarios (S1–S5) are calibrated differently and for a different purpose. For S1–S4 the threshold m is placed close to the baseline mean μ rather than at an extreme. This is necessary for the three-method comparison to be informative: a threshold far below μ makes success trivially certain, one far above makes it trivially impossible, and the *upper-tail* Chernoff form is not defined when $m \leq \mu$ — the gap that Section 3.8 closes with a lower-tail certificate. S5 is the deliberate exception — a threshold far beyond what its budget can reach — and functions as a negative control: a model that reported S5 as succeeding would be wrong in an obvious way.

6.6 Paired randomised design

The core experiment is a **matched-pairs design**. Each trial draws one scenario and evaluates it twice, under both configurations, with identical N, m, B, R, P, S, A and identical random streams. Only the acceptance and detection probabilities differ.

The alternative — drawing independent samples for each arm — would confound the countermeasure effect with the difference between the scenarios happening to be drawn for each arm. Because the per-trial success probability has a standard deviation of 0.474 for bribery, scenario-to-scenario variance dominates the effect being measured, and an unpaired design would need far more trials to reach the same precision. Pairing removes that variance entirely: the reported reduction is a within-scenario effect, and every pair is its own control.

Inference. Confidence intervals are computed by bootstrap resampling of the paired differences (10,000 resamples) rather than by a normal approximation. This is not a stylistic preference. The per-trial success probabilities are strongly bimodal — most scenarios are either comfortably safe or comfortably lost — which is visible in the summary statistics: the baseline bribery distribution has a mean of 0.350 but a median of 0.0000. A distribution with that shape violates the assumptions of a t-interval, and the bootstrap makes no distributional assumption.

Trial count. 2,500 pairs per vector were run, above the 1,000 originally specified, chosen so that the bootstrap intervals on the mean reduction are narrow relative to the effect: the bribery reduction interval $[0.269, 0.304]$ has a width under 0.04 against an effect of 0.287.

6.7 Triangulation of three computational methods

Every trial computes its success probability three ways — exact binomial, Monte Carlo, and Chernoff bound where valid — on identical inputs. This is methodological triangulation, and each method plays a distinct role.

Exact binomial.
The ground truth. Every reported probability is this value; the other two exist to check it and to characterise their own usefulness.

Monte Carlo.
An independent numerical path to the same quantity, through a different code path (sampling rather than the survival function). Agreement is evidence that neither contains an error; its error against the exact value quantifies how much simulation is needed for a given precision (RQ2).

Chernoff bound.
A provable upper bound. Its role is not accuracy but *certification*: a margin the bound rules out is ruled out regardless of simulation effort. Its looseness is measured and reported rather than treated as a defect.

The three are computed on the same trial, not on separate runs, which is what makes the comparison a validation rather than a coincidence. Critically, the bound is applied only where mathematically valid: the

dispatcher determines whether m lies in the upper tail, the lower tail, or neither, and returns no bound with a machine-readable reason outside those regions. Averaging the three quantities across heterogeneous trials — as an earlier iteration of this work did — compares them on different sub-populations, since the bound exists only where $m > \mu$; Figure 1 therefore holds every other parameter fixed and sweeps only q.

6.8 Threshold solver design

The threshold solvers (T1–T7) invert the model, and three design decisions govern their trustworthiness.

Non-existence is a result. Each solver returns a structured object carrying either a value or an explicit absence with a machine-readable reason. No solver substitutes an endpoint, a zero or a sentinel for a threshold that does not exist, and the figures print "does not exist" rather than drawing a line. A threshold silently defaulted to a domain boundary would be indistinguishable, in a plot, from a real finding.

Exactness where the domain is discrete. T3, T6 and T7 search over integers and use binary search, which is exact because each underlying function is monotone in the search variable: $\mathbb{P}(X \geq m)$ is non-decreasing in n, and the Chernoff bound is decreasing in m above the mean. These are not approximations.

A definitional choice, stated. T4 and T5 required resolving an ambiguity in what "the attacker's utility" means. The headline optimiser searches $T \geq 0$ and therefore includes declining to attack, so its optimum is never negative — and a quantity floored at zero cannot be bisected for a zero crossing. The thresholds are consequently defined on $\max_{T \geq 1} U(T)$, the signed payoff of actually attacking, which goes negative exactly when attacking loses money. This is a definition rather than a derivation, it changes the reported numbers, and it is stated wherever those thresholds appear.

6.9 Coalition search design

The coalition analysis searches the budget simplex exhaustively at a 0.05 step (231 allocations for three vectors) rather than using continuous optimisation. The reason is structural: the objective is piecewise flat wherever $\lfloor B/c \rfloor$ does not change, so gradients are zero almost everywhere and undefined at the steps, which defeats gradient methods. Exhaustive evaluation is affordable because each evaluation is a convolution rather than a simulation. The reported optimum is the best grid point and may differ slightly from the true continuous optimum — stated as a limitation rather than glossed.

The success probability of a coalition is computed exactly. A sum of binomials with unequal success probabilities is not binomial and has no closed form, and the obvious approximations (normal, Poisson) are least accurate precisely in the tail that matters. Convolving the component PMFs is exact, and truncating each at $m-1$ before convolving is also exact rather than approximate, since $\mathbb{P}(W \geq m) = 1 - \mathbb{P}(W \leq m-1)$ and no combination summing to at most $m-1$ can involve a component larger than $m-1$. That truncation is what makes national-scale margins tractable: it holds the transform length at m rather than at the total attempt count, which is two orders of magnitude larger.

6.10 Validation and verification

Correctness is established at four levels.

Unit level (122 tests). Beyond routine coverage, several tests encode properties the mathematics requires and that a plausible-looking implementation can silently violate: that the lower-tail Chernoff bound carries a negative exponent, since a positive one yields a "bound" exceeding 1 that is vacuous but looks reasonable in output; that the bound is never returned outside its region of validity; that $T = 0$ rather than a division error when $B = 0$, $R = 0$ or $B < R$; and that each threshold solver's answer actually brackets the crossing it claims to find.

Cross-method level. The exact and Monte Carlo values are compared on every trial; the Chernoff bound is checked against the exact value on every trial where it is defined. A violation would appear as a bound lying below the exact value it is supposed to bound (Figure 3); none occurs across 1,088, 3,471 and 3,604 valid trials respectively.

Cross-layer level. The abstract probabilities and the concrete mined-election flip rates are compared for ordering and magnitude (Section 8.5).

Reproducibility level. A test asserts that two runs at the same seed produce identical output, and a second asserts that different seeds do not, so the first cannot pass vacuously.

6.11 Figure geometry on a discrete support

Every random variable in this model is integer-valued: surviving fraudulent votes, attempts, and margins are counts. That is a property of the model with a consequence for how its distributions are drawn, and it is recorded here because a figure can be numerically exact and still mislead — a failure mode no numerical test detects.

Plotting a discrete probability mass function as unit-width centred bars places the bar for outcome k on the interval $[k-0.5, k+0.5]$. The event $\{X \geq m\}$ therefore occupies the region whose left edge lies at $m-0.5$, not at m. Drawing the labelled threshold line at m places it through the *centre* of the first bar of the success region, so the colour change appears to begin half a vote before the threshold it is labelled with, even though the shaded probability mass is exactly correct. An earlier draft of Figure 7 contained precisely this defect. It was found visually in review rather than by any test, which is why the three tests that now pin it were written afterwards.

The convention adopted throughout is that boundaries between integer outcomes are drawn at half-integers and labelled with the integer margin they separate: the manipulation threshold at $m-0.5$, the reachable margin m_α at $m_\alpha + 0.5$ (since margins at or *below* m_α are reachable), and the certified margin m_ε at $m_\varepsilon - 0.5$ (since margins at or *above* m_ε are certified). Quantities that are genuinely continuous are not moved: the mean $\mu = T(1-q)$ is an expectation rather than a boundary between outcomes, and is drawn at its true coordinate.

The correction changes no probability, threshold, table or finding in this report: the shaded mass of Figure 7 agreed with $P(X \geq m)$ to machine precision both before the correction and after it. Three tests in `tests/test_figure_geometry.py` now pin the geometry by interrogating the plotting library's

rectangle and line coordinates directly, so the visual property is verified in the same way as the numerical ones rather than left to the eye.

6.12 Reproducibility protocol

All randomness derives from a single seeded `numpy.random.Generator`; no global random state is used anywhere, and no unseeded standard-library random calls exist in the codebase. Re-running with seed 42 reproduces every number in this report bit-for-bit — verified by regenerating the full pipeline from a clean results directory and comparing the summary output. The threshold and coalition analyses are fully analytic and require no seed at all.

The report itself is part of the protocol. Every *experimental result* in the prose is injected at build time from `results/json/` and `results/csv/` as a LaTeX macro rather than typed. A macro whose experiment has not been run causes the document to fail compilation rather than to print a stale value, so the text cannot silently disagree with the data it describes. The exceptions are deliberate and are not results: configuration values quoted from `config.py`, worked-example arithmetic shown at reduced precision so that it reads on the page, and the analytic counter-examples of Section 3.7, which are computed and regression-tested in `tests/test_chernoff.py` rather than in the results pipeline.

6.13 Ethical considerations

This work models attacks on elections, which warrants explicit treatment.

No real system, election, voter or personal datum is involved at any point. All identifiers are simulation-internal indices; the candidates are *A* and *B* and have no referent. The artefact is a mathematical model, not a tool: it computes probabilities and contains no capability that could be directed at a deployed system, and nothing in it constitutes an exploit, a technique for evading a specific control, or an implementation of one.

The disclosure balance favours publication. The vectors modelled — bribery, Sybil registration, ballot stuffing — are long-established in the literature reviewed in Section 2, and the contribution is a quantification of their effect and of the countermeasure strength required to blunt them. That quantification is of substantially more use to a defender, who can act on a detection threshold, than to an attacker, who learns only that the attacks they already knew about work about as well as expected.

7 The Experiment in Depth

This section describes exactly what was run. Section 7.2 traces one real trial end to end with its actual numbers, and Sections 7.3–7.7 give the protocol for each of the five experiments.

Table 3. Randomised parameter ranges. Every trial draws one value per symbol uniformly from these ranges; the protected configuration modifies only the acceptance and detection probabilities, never N, m, B, R, P, S or A.

Symbol	Meaning	Range
N	Electorate size	200 – 5,000
m	Manipulation threshold	2% – 25% of N
B	Attacker budget	1,000 – 200,000
R	Bribe reward per voter	10 – 500
P	Penalty if a bribed voter is caught	10 – 2,000
q	Bribery detection probability (baseline)	0.02 – 0.25
q	Bribery detection probability (protected)	baseline + 0.15 – 0.60
S	Attempted forged identities	10 – 1,000
q_s	Registration acceptance (baseline)	0.40 – 0.95
p_d	Sybil detection (baseline)	0.02 – 0.30
A	Attempted fraudulent ballots	10 – 1,000
q_b	Ballot acceptance (baseline)	0.30 – 0.90
p_b	Ballot detection (baseline)	0.02 – 0.30
q_s	Registration acceptance (protected)	baseline $\times$ (1− 0.30 – 0.80)
p_d	Sybil detection (protected)	baseline + 0.20 – 0.70
q_b	Ballot acceptance (protected)	baseline $\times$ (1− 0.30 – 0.80)
p_b	Ballot detection (protected)	baseline + 0.20 – 0.70

Note. Source: `config.py::PARAM_RANGES`. Currency-like quantities are unitless in the model; only ratios such as B/R affect the results.

Table 4. The five experiments, what each isolates, and what it produces.

#	Experiment	Isolates	Produces
E1	Paired randomised trials	Countermeasure effect, holding the scenario fixed	2,500 matched pairs per vector; RQ1–RQ3
E2	Concrete mined elections	Whether abstract probabilities correspond to real winner flips	300 mined elections per vector; ground truth
E3	Benchmark scenarios	Behaviour at five interpretable, fixed operating points	Quotable per-scenario numbers; a negative control
E4	Threshold solving	The parameter value at which each attack stops being viable	Seven thresholds per scenario; RQ4
E5	Coalition and sensitivity	Whether combining vectors beats the best single vector, and whether that survives the cost assumptions	231-point simplex search; 108-configuration sweep; RQ5

7.1 The five experiments

7.2 Anatomy of a single trial

The clearest way to describe the experiment is to follow one trial through it. Trial 79 is used below. It is an ordinary trial drawn by the sampler, not a constructed illustration, and it was chosen because it happens to exhibit all three of the behaviours the model is built to distinguish.

Step 1 — Draw the scenario

The generator draws one scenario from the ranges of Table 3, using the trial's own stream from the seeded generator:

$N = 4,458$ voters	$m = 261$ votes	$B = 122,592$	$R = 372.81$
$P = 1051.88$	$S = 981$ identities	$A = 104$ ballots	

The manipulation threshold is $m = 261$: the attack succeeds if and only if at least that many fraudulent votes survive every countermeasure and reach the count. Every number above is now *fixed* for both arms of this pair. Only the acceptance and detection probabilities will differ.

Step 2 — Bribery: the feasibility constraint

The attacker can approach at most

$$T = \min\left(N, \left\lfloor \frac{B}{R} \right\rfloor\right) = \min\left(4,458, \left\lfloor \frac{122,592}{372.81} \right\rfloor\right) = 328 \text{ voters},$$

so the budget binds well before the electorate does. Note already that $T = 328$ exceeds $m = 261$ — the attack is *arithmetically* possible — but only just, which is what makes the survival probability decisive.

Step 3 — Bribery: the rational-voter gate

This is where the two arms diverge, and it is the most consequential step in the model.

Baseline arm, $q = 0.229$:

$$U_v = (1-q)R - qP = (1-0.229)(372.81) - (0.229)(1051.88) = 45.85 > 0 \;\Rightarrow\; \text{voter } accepts.$$

Protected arm, $q = 0.517$:

$$U_v = (1-0.517)(372.81) - (0.517)(1051.88) = -364.10 < 0 \;\Rightarrow\; \text{voter } rejects.$$

The attacker's budget, the electorate and the margin are identical in both arms. The bribe is simply no longer worth accepting. Under the protected configuration the attack contributes zero fraudulent votes not because they were detected and removed, but because they were never cast.

Step 4 — Bribery: distribution and success probability

In the baseline arm each bribed vote survives with probability $1 - q = 0.771$, so

$$X \sim \text{Binomial}(328,\ 0.771), \qquad \mu = \mathbb{E}[X] = 252.7.$$

The mean $\mu = 252.7$ sits *below* the threshold $m = 261$, so the attack succeeds only on an upper-tail excursion. The exact probability is

$$\mathbb{P}(X \geq 261) = 0.153.$$

Three methods are now applied to this same quantity:

Method	Value	Role
Exact binomial survival function	0.153	ground truth
Monte Carlo, 2,000 draws	0.160	independent numerical check
Chernoff upper bound (M7)	0.875	provable ceiling

All three orderings that the mathematics requires hold: the Monte Carlo estimate is close to the exact value, and the bound lies above it. The bound is defined here precisely because $m > \mu$. In the protected arm the gate is closed, so $\mathbb{P} = 0.000$ and the bound is not applicable — correctly reported as absent rather than as zero, since no bound was computed.

Step 5 — Sybil: two gates in series

The same trial, same threshold $m = 261$, with $S = 981$ forged identities attempted. A forged identity must pass registration *and* escape detection to place a counted vote:

$$W \sim \text{Binomial}\big(S,\ q_s(1 - p_d)\big).$$

Baseline, $q_s = 0.771$, $p_d = 0.274$: $\mathbb{E}[W] = 549.3$, which is more than twice the threshold, so $\mathbb{P}(W \geq m) = 1.000$ — effectively certain.

Protected, $q_s = 0.425$, $p_d = 0.705$: $\mathbb{E}[W] = 122.8$, now well below the threshold, so $\mathbb{P}(W \geq m) = 0.000$.

The mechanism differs from bribery's. Nothing refuses; the two gates simply tighten, the mean crosses from one side of the threshold to the other, and the probability follows it sharply because the binomial concentrates around its mean.

Step 6 — Ballot stuffing: structural infeasibility

The same trial attempts $A = 104$ fraudulent ballots against a threshold of $m = 261$. Since $A < m$, the attack cannot reach the threshold even if every single ballot were accepted and undetected:

$$\mathbb{P}(W \geq 261) = 0.000 \text{ (baseline)}, \qquad 0.000 \text{ (protected)}.$$

The expected surviving counts — 59.1 and 27.2 — confirm how far short the attack falls. This is not a detection failure but a capacity limit, and it is the phenomenon that Section 8.7 shows to be escapable by combining vectors.

What this one trial demonstrates

> **Three distinct failure mechanisms, in one trial**
>
> Trial 79 shows the model distinguishing three different reasons an attack fails. Bribery fails in the protected arm because the bribe stops being *worth accepting* ($U_v = -364.10$). The Sybil attack fails because the expected surviving count crosses below the threshold ($549.3 \rightarrow 122.8$ against $m = 261$). Ballot stuffing fails in both arms because fewer ballots are attempted than the threshold requires ($A = 104 < m = 261$) — a structural limit no detection setting affects. A model reporting only aggregate success rates would collapse these three into one number and lose the distinction that tells a defender what to fix.

7.3 Experiment 1: paired randomised trials

Procedure. Repeat the trial above 2,500 times per vector, drawing a fresh scenario each time and evaluating both arms. Per trial, record: the drawn parameters; the rational-voter decision; T; the exact, Monte Carlo and Chernoff values; expected surviving counts; detection probabilities; and attacker cost.

Output. 5,000 rows per vector (baseline and protected for each of 2,500 pairs), 15,000 rows in the combined Chernoff-validation table, and an aggregate summary.

Analysis. Paired differences are aggregated into means with bootstrap confidence intervals (10,000 resamples), and the exact value is compared against the Monte Carlo estimate and the Chernoff bound trial by trial.

What it controls for. Because both arms share the scenario, any difference is attributable to the configuration alone. What it does *not* control for is the choice of parameter ranges, which is why the ranges are broad and why the benchmark scenarios of E3 exist as a complementary fixed-point analysis.

7.4 Experiment 2: concrete mined elections

E1 produces probabilities. E2 produces winners.

Procedure. For each of 300 trials per vector:

1. Draw an electorate of 150–900 voters and a support level for candidate B in $[0.35, 0.65]$, biased toward close races so that the manipulation threshold is small enough for an attack to be interesting.
2. Every legitimate voter casts a ballot. The batch is sealed into a genuinely mined block: proof-of-work at difficulty 3, meaning the block hash must begin with three zeros, found by incrementing a nonce until it does.
3. Read the pre-attack tally *from the chain* via `count_votes()` — not from a Python-side running total.

This matters: the tally under test is the ledger's own.

4. Compute the flip threshold m from the actual margin (Equation (1)) and identify the losing candidate as the attacker's target.
5. Simulate each fraudulent attempt individually through the real pipeline: an acceptance gate (bribe accepted / registration passed / ballot validated), then a detection gate. Only votes that are accepted *and* undetected are mined into the chain. Rejected and detected attempts are logged but never reach the ledger, which is what a real countermeasure does — a rejected transaction is not a recorded vote.
6. Read the post-attack tally from the chain, determine the winner, and record whether it changed.
7. Run `is_chain_valid()` to confirm the ledger is still internally consistent: every hash correct, every link intact, every block satisfying the difficulty target.

Both arms run against *the same starting election*, so the pairing of E1 is preserved: the two configurations are handed an identical mined chain and an identical margin.

What it establishes. Two things. First, that the abstract probabilities correspond to real outcomes rather than to an artefact of the abstraction. Second — and this is the finding the dissertation turns on — that chain integrity and election correctness are independent: the validity check passes in every trial, including the ones where the winner changed.

7.5 Experiment 3: benchmark scenarios

E1 averages over a distribution of scenarios, which is statistically powerful and hard to reason about concretely. E3 fixes five interpretable operating points — a university election ($N = 100$), an organisation election ($N = 10,000$), a national election ($N = 100,000$), a maximum-capability high-budget attack, and a deliberately under-resourced low-budget attack — and evaluates each exactly, by Monte Carlo, by Chernoff bound and by the game-theoretic optimiser.

S5 is a negative control. Its threshold is set far beyond what its budget can reach, and the model is expected to report it as failing; a model that reported S5 succeeding would be wrong in a way that no amount of aggregate statistics would reveal. It reports $P = 0.000$.

7.6 Experiment 4: threshold solving

For each benchmark scenario, the seven thresholds of Section 4 are solved.

Closed-form thresholds (T1, T2) are evaluated directly. T2 is also verified against the numerically computed zero contour of the utility surface, and the two coincide (Figure 2).

Integer thresholds (T3, T6, T7) are found by binary search over a monotone function, which is exact. For T6 the search runs over margins above the mean, where the Chernoff bound decreases monotonically.

Continuous thresholds (T4, T5) are found by bisection on the signed utility of attacking. Each solver first checks that the endpoints bracket a crossing, and returns an explicit absence if they do not — which is what happens for B^* under the protected configuration, where no budget makes the attack profitable

because the bribe is not incentive-compatible at any budget.

Two supporting sweeps generate the data behind Figure 6 and the break-even budget: utility against detection probability across $q \in [0,1]$, and utility against budget up to three times the scenario budget, both for every scenario and both arms.

7.7 Experiment 5: coalition and sensitivity

Coalition search. For each scenario and arm, the three vectors are placed on a common budget line by assigning unit costs: a bribe costs R; a forged identity costs 40% of R; an injected ballot costs 15% of R. Structural ceilings apply — bribery at N by Equation (4), forged registrations at 20% of N, injected ballots at 30% of N. The budget simplex is then evaluated exhaustively at a 0.05 step, 231 allocations, each scored by the exact convolution of the three component distributions. The best allocation is compared against the best pure single-vector strategy, which is a corner of the same simplex.

Sensitivity sweep. The four constants above have no empirical basis, so the entire coalition analysis is re-run across their product: identity cost in $\{0.2, 0.4, 0.8\}$, ballot cost in $\{0.05, 0.15, 0.40\}$, registration ceiling in $\{0.10, 0.20\}$ and ballot ceiling in $\{0.15, 0.30\}$, for three scenarios — 108 configurations in total. For each, the analysis records whether the coalition still beats the best pure strategy.

This is the check that decides whether the coalition finding is a result or an artefact of convenient parameters, and it is reported as such in Section 8.7.2.

8 Findings

This section reports what the five experiments produced, organised by finding rather than by experiment. Each subsection states the finding, gives the evidence, and says what it rests on. Table 5 summarises the six.

Findings F1–F4 answer the core research objectives: model the three attacks, implement countermeasures, and demonstrate experimentally whether the countermeasures reduce attack success. F5 and F6 go beyond those objectives. Having established that the countermeasures work, the threshold analysis asks the inverse question — how much countermeasure is actually *required* — and the coalition analysis asks whether evaluating vectors one at a time understates the achievable attack. Both are extensions to the original scope rather than part of it, and they are reported as such.

8.1 Overview

Table 6 gives the headline comparison across the 2,500 matched pairs per vector. All three vectors fall sharply under the protected configuration, and every bootstrap confidence interval on the paired reduction excludes zero comfortably.

Table 5. Summary of findings.

#	Finding	Principal evidence
F1	Detection is a sharper lever than budget; success collapses over a narrow band of q rather than declining smoothly	E1 budget and detection sweeps; Figure 1
F2	Monte Carlo reproduces the exact value to $\sim 10^{-4}$; the Chernoff bound never failed but is loose by 0.0423	E1 triangulation; Figure 3
F3	Countermeasures cut success probability by 71.8–81.9%, but residual risk concentrates in identity verification	E1; Table 6, Figure 4
F4	Chain integrity and election correctness are independent: the ledger was never violated and elections were stolen anyway	E2; Figure 5
F5	Attacks stop paying at detection far below what full protection delivers: deterrence is roughly 4.7$\times$ cheaper than prevention	E4; Table 7, Figure 6
F6	Combining vectors beats the best single vector in 4 of 5 scenarios, where two of three are individually infeasible	E5; Table 9, Figure 10

Table 6. Headline experimental results across 2,500 paired randomised trials per vector.

Attack	Baseline	Protected	Absolute red.	Relative red. (%)	95% CI	MC error	Bound holds (%)
Bribery	0.350	0.064	0.287	81.859	[0.269, 0.304]	8.0e-05	100.000
Sybil	0.480	0.136	0.345	71.779	[0.327, 0.362]	4.0e-04	100.000
Ballot Stuffing	0.442	0.116	0.325	73.695	[0.308, 0.343]	3.9e-04	100.000

Note. Confidence intervals are bootstrap intervals on the mean paired reduction (10,000 resamples). "Bound holds" is the percentage of trials with a mathematically valid Chernoff region in which the bound was not violated.

Which probability is being reported

Two of the three vectors pass fraudulent votes through *two* independent gates, so they admit two different success probabilities, and the distinction has to be made explicit before any table is read.

- $P(\text{accepted} \geq m)$ — the probability that at least m fraudulent votes clear the *acceptance* gate (Sybil registration, or ballot admission). This is an intermediate quantity: it ignores detection entirely, is always the larger of the two, and is not comparable across vectors.
- $P(\text{surviving} \geq m)$ — the probability that at least m fraudulent votes clear *both* gates, being accepted *and* escaping detection. This is the headline measure. Every figure, every stated finding and every conclusion in this chapter uses it.

Bribery has a single effective gate, because detection is already folded into its per-attempt success probability $1-q$; for bribery the two measures coincide. For Sybil and ballot stuffing they do not, and the gap is substantial — under the baseline configuration the Sybil acceptance-only figure is 0.537 against a both-gates figure of 0.480, and the corresponding ballot-stuffing pair is 0.498 against 0.442.

Both quantities are reported, because the difference between them is itself a result: it isolates how much of the countermeasure effect comes from tightening registration and admission rather than from detection. The per-vector tables therefore carry an explicit `measure` column, and the aggregate tables report the headline measure only, so that the numbers a reader is most likely to quote are the same in every table. A test asserts this agreement (`tests/test_reporting_consistency.py`), because the two measures were at one point reported side by side without being named, which made correct numbers look inconsistent.

> **F3 — effect of countermeasures (RQ3)**
>
> Across 2,500 matched pairs, the protected configuration reduced mean attack success probability from 0.350 to 0.064 for bribery (81.9% relative), from 0.480 to 0.136 for Sybil attacks (71.8%), and from 0.442 to 0.116 for ballot stuffing (73.7%). The 95% bootstrap interval for the bribery reduction is $[0.269, 0.304]$.

One feature of the distributions is worth stating before the detail, because it governs how the means should be read. The per-trial success probabilities are strongly bimodal: the baseline bribery distribution has a mean of 0.350 but a median of 0.0000 and a standard deviation of 0.474. Most scenarios are either comfortably safe or comfortably lost. A mean of 0.350 therefore does not describe a typical election facing a 35% risk; it describes a population in which roughly a third of elections are in serious trouble and the rest are not. This is why the analysis reports thresholds (F5) rather than resting on averages.

8.2 F1 — Budget and detection are not symmetric levers

Bucketing the 2,500 trials by attacker budget shows success probability rising steadily with budget under both configurations. The baseline curve crosses even odds inside the sampled range; the protected curve never does, at any budget tested.

The budget effect is gradual and saturating: additional money buys additional attempts only until the capacity ceiling $T = \min(N, \lfloor B/R \rfloor)$ binds, after which it buys nothing. There are two distinct infeasibility regimes: below the $T = m$ contour the attacker cannot reach the threshold even with a perfect success rate, and above the $T = N$ contour the electorate is exhausted, so further budget buys nothing at all.

The detection effect has a different shape entirely. Figure 1 holds every other parameter fixed and sweeps only q: success stays near certainty until the survival probability $1 - q$ falls far enough that the mean drops below the threshold, at which point it collapses over a narrow band. Detection does not erode an attack; past a point, it ends it.

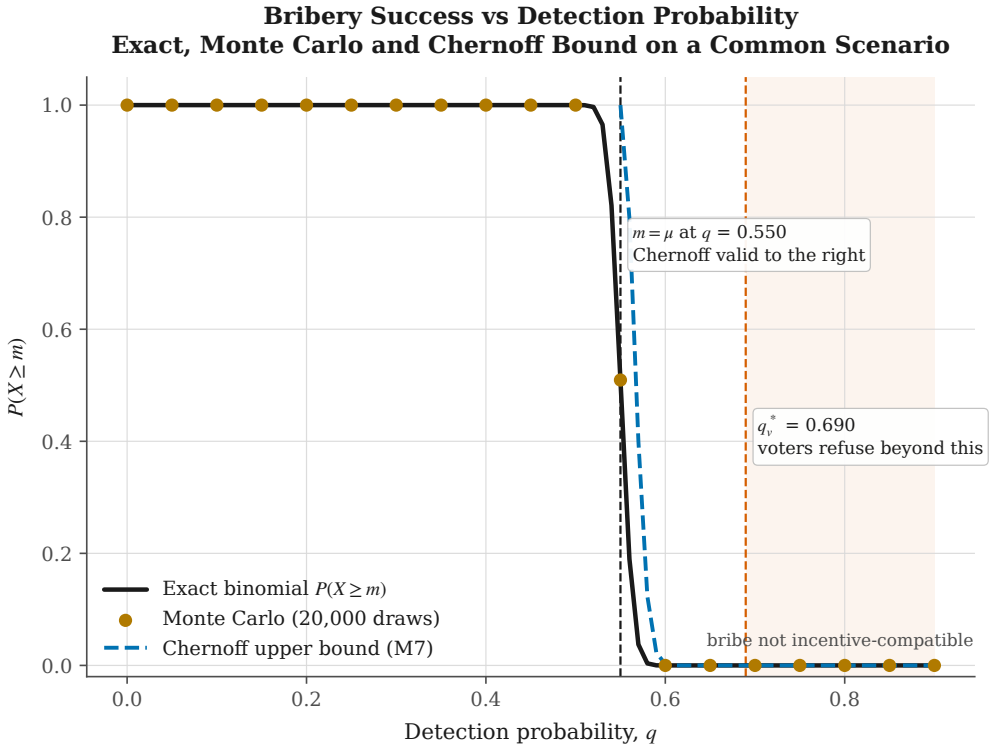


Figure 1. Exact binomial, Monte Carlo and Chernoff bound against detection probability on one fixed scenario. Averaging these three quantities across heterogeneous randomised trials would compare them on different sub-populations — the bound exists only where $m > \mu$ — so this figure varies only q.

Two mechanisms produce that collapse and they compound. The mean surviving count $T(1-q)$ falls linearly in q, and the binomial concentrates tightly around its mean, so once the mean crosses the threshold the tail probability falls away exponentially. On top of that, bribery carries the rational-voter gate: beyond $q_v^* = R/(R+P)$ the bribe is refused outright. Figure 2 shows that surface, with the closed-form threshold traced over the numerically computed zero contour. Both are evaluated by the production modules — the surface by the M1 implementation, the curve by the T2 solver — so their coinciding checks that the solver returns the root of the utility function as coded. It is a check on the implementation, not independent evidence for the model: T2 is the analytic root of M1, so agreement is expected, and only a coding error could separate the two.

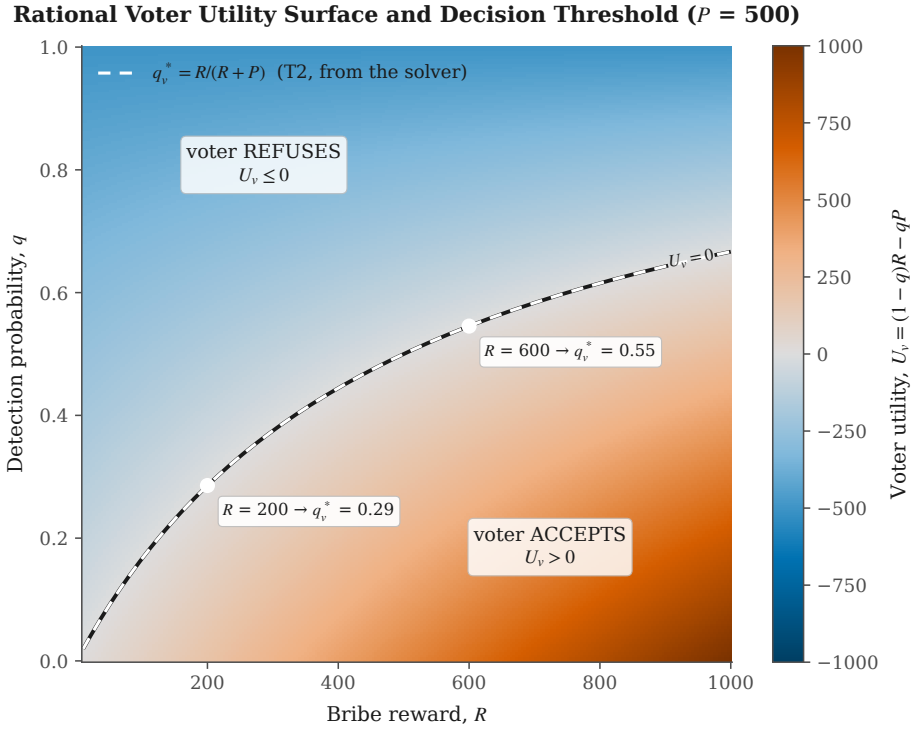


Figure 2. Rational voter utility $U_v = (1-q)R - qP$, with the $U_v = 0$ decision boundary and the closed-form indifference threshold $q_v^* = R/(R+P)$ drawn together.

F1 — where defensive effort pays (RQ1)

Budget and detection are not interchangeable. Budget raises success gradually and saturates at a capacity ceiling that no further spending moves. Detection produces a sharp collapse, because it lowers the mean and — for bribery — eventually removes the attacker's supply of willing voters altogether. A defender choosing between raising the cost of an attack and raising the chance of catching it should prefer the latter.

8.3 F2 — The three methods agree, and each has a distinct use

RQ2 asks how well the Monte Carlo estimate and the Chernoff bound reproduce the exact value.

The Monte Carlo estimator is accurate: mean absolute error against the exact binomial probability was 7.98e-5 for bribery, 3.95e-4 for Sybil and 3.94e-4 for ballot stuffing, with worst-case errors of 0.0149, 0.0305 and 0.0230 respectively. A convergence sweep over sample sizes from 10 to 50,000 draws (`results/csv/monte_carlo_convergence.csv`) shows the estimator settling to within 0.01 of the exact value from 3,000 draws onward, reaching an absolute error of 0.0018 at the largest sample size. The 2,000 draws used per scenario therefore sit inside the converged regime. A two-standard-error envelope is the looser test here and is reported in the same file: since it is a 95% interval, isolated excursions are expected and two of the nine sample sizes show one, which is why the settling point is quoted against a fixed tolerance rather than against the envelope.

The Chernoff bound held in every trial where it was mathematically defined: 100.0% of 1,088 valid

bribery trials, 100.0% of 3,471 Sybil trials and 100.0% of 3,604 ballot-stuffing trials, with no violation anywhere. It is also loose — a mean gap of 0.0423 for bribery — which is the expected behaviour of a bound rather than a defect.

The bound is one-sided, however, and it exists only where the threshold sits above the mean. That left the scenarios with $m \leq \mu$ — the ones in which the attack is *most* likely to succeed — with no analytic statement at all. The two-sided certificate of Section 3.8 closes that gap by applying the $(1-\delta)$ lower-tail form to the failure event, and Figure 3 validates both directions against the exact value on the same experiment.

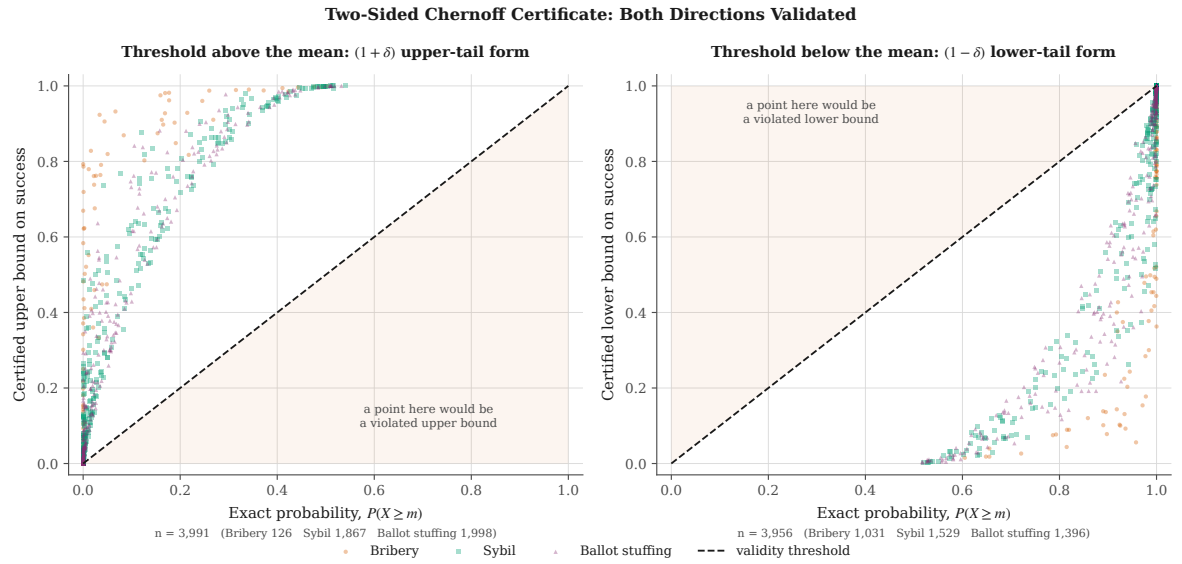


Figure 3. Both Chernoff directions validated. Left: threshold above the mean, where the $(1+\delta)$ upper-tail form certifies success from above — valid points lie on or above the diagonal. Right: threshold below the mean, where the $(1-\delta)$ lower-tail form applied to the failure event certifies success from below — valid points lie on or below it. No point falls on the wrong side of either diagonal.

With the certificate in place, every one of the 15,000 scenario evaluations carries a guaranteed statement, and the certified interval contains the exact value in 100% of them.

F2 — when to use which method (RQ2)

Use the exact binomial value whenever it is affordable; it always was here, even at $N = 100,000$. Use Monte Carlo where the quantity has no closed form — the concrete election simulation, for instance. Use the Chernoff bound when a *provable* statement is required rather than an estimate: it is the only one of the three that certifies a margin as safe independently of how the simulation ran, which is what makes the certified margin m_ε meaningful as a security claim rather than an observation.

8.4 F3 — Countermeasures work, but not uniformly

Both remaining vectors behave more smoothly than bribery, because neither has a consent gate: a forged identity does not decline. Their success probability rises steadily with effort rather than switching off at a threshold.

Sybil attacks are the most resilient of the three under the tested parameter distribution, retaining a mean success probability of 0.136 under protection — roughly twice the protected bribery figure of 0.064. The reason is structural rather than numerical: bribery is suppressed twice over, once by detection and once by the voter's refusal, whereas a Sybil attack is suppressed only by detection. Figure 4 shows the three vectors together.

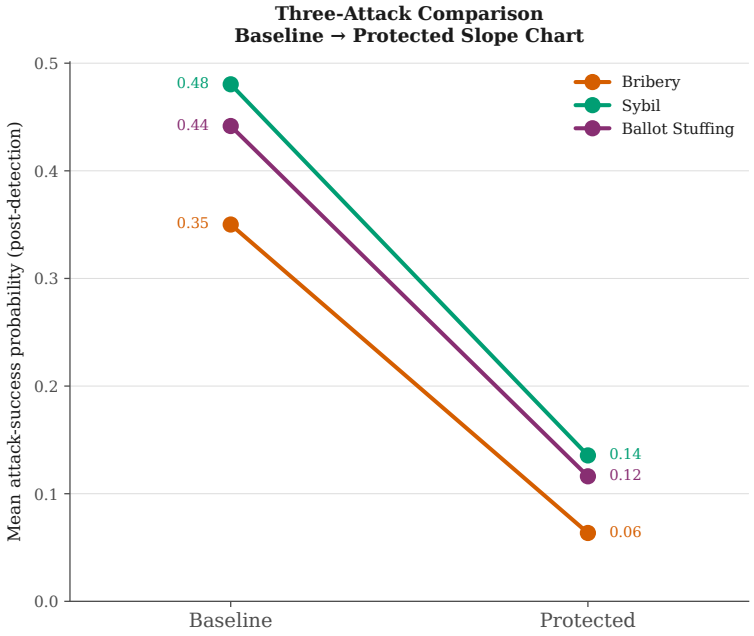


Figure 4. Mean success probability for the three vectors, baseline against protected.

The distributions add something the means conceal: under protection they are compressed toward zero rather than merely shifted downward. The countermeasures remove the upper tail of highly successful attacks, which is the part of the distribution that actually decides elections. They do not remove the attack. Under protection the Sybil and ballot-stuffing vectors retain mean success probabilities of 0.136 and 0.116 respectively, so the defensible claim is that attack success is *substantially reduced* under the modelled countermeasures, not that it is eliminated.

> **F3b — where the residual risk sits**
>
> Identity verification, not vote recording, is the binding constraint. Sybil attacks remain the most successful vector under protection (0.136 mean success probability) because the ledger's integrity guarantees say nothing about whether the identity behind a ballot is real. This is consistent with voter authentication being repeatedly flagged as an open problem in the systematic reviews [16, 18]. A deployment that hardens the chain while leaving registration weak has hardened the wrong layer.

8.5 F4 — Chain integrity and election correctness are independent

The results so far are probabilities. Figure 5 reports what happened when 300 elections were genuinely mined on a proof-of-work chain, attacked, and re-tallied from the chain to see who actually won.

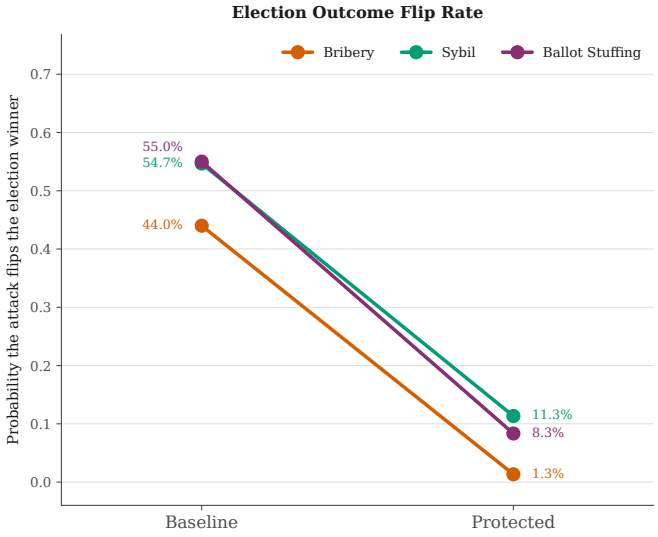


Figure 5. Probability that an attack flips the winner of a concrete, blockchain-recorded election.

Baseline flip rates were 44.0% for bribery, 54.7% for Sybil and 55.0% for ballot stuffing, falling under protection to 1.3%, 11.3% and 8.3%. This is the cross-layer check that Section 6.2 exists to provide: two independent implementations — one analytic, one a mined chain — agree in magnitude, in the direction and size of the protection effect, and in placing bribery lowest in both layers. They do not agree on the ordering of the top two: the analytic layer puts Sybil (0.480) above ballot stuffing (0.442), while the concrete layer reverses them, at 54.7% against 55.0%. That gap is a third of a percentage point on 300 trials per vector — well inside sampling error — so the two vectors are better read as indistinguishable at the concrete layer than as ranked.

Individual trials make this concrete. For a representative flipped baseline election, the attacker's surviving fraudulent votes carry the target past the opponent's tally, while the protected run on the *same* starting election leaves the original winner in place.

> **F4 — the central observation**
>
> Chain integrity was never violated in any of the 300 trials. In the 44.0% of baseline bribery trials that flipped the winner, the blockchain was working perfectly: every fraudulent vote was correctly formed, correctly mined and permanently recorded, and the validity check passed afterwards. Immutability guarantees that a wrong result cannot subsequently be altered. It does not guarantee that the result is right. This is the quantitative form of the argument Park et al. [17] make qualitatively.

8.6 F5 — Deterrence is far cheaper than prevention

Table 7 collects every threshold for the five benchmark scenarios. The critical detection probability q_d^* — the point at which mounting the attack stops paying — lies between 0.050 and 0.118 across the 4 scenarios where detection is the binding constraint. The protected configuration operates at $q = 0.55$, roughly 4.7 times higher than the strongest of those requirements.

Table 7. Critical thresholds for the five benchmark scenarios under the baseline configuration.

Scenario	N	m	R^*	q_v^*	$T_{\min}$	q_d^*	B^*	T^*	m_α	m_ε	$P(X \geq m)$
S1	100	35	52	0.118	38	0.118	2,880	37	34	—	0.423
S2	10,000	1,943	68	0.118	2,089	0.072	249,480	2,083	1,937	—	0.328
S3	100,000	19,015	63	0.111	20,016	0.050	2,998,650	20,000	19,000	19,515	0.320
S4	5,000	4,758	26	0.167	5,008	0.051	499,400	5,000	4,750	—	0.316
S5	5,000	550	26	0.167	579	0.000	57,100	0	76	—	0.000

Note. R^* minimum bribe a rational voter accepts (T1); q_v^* voter-indifference detection (T2); $T_{\min}$ attempts for even odds (T3); q_d^* critical detection at which attacking stops paying (T5); B^* break-even budget (T4); T^* utility-maximising attempts; m_α margin beaten at even odds (T7); m_ε Chernoff-certified safe margin at $\varepsilon = 10^{-3}$ (T6). A dash means the threshold does not exist in the searched domain.

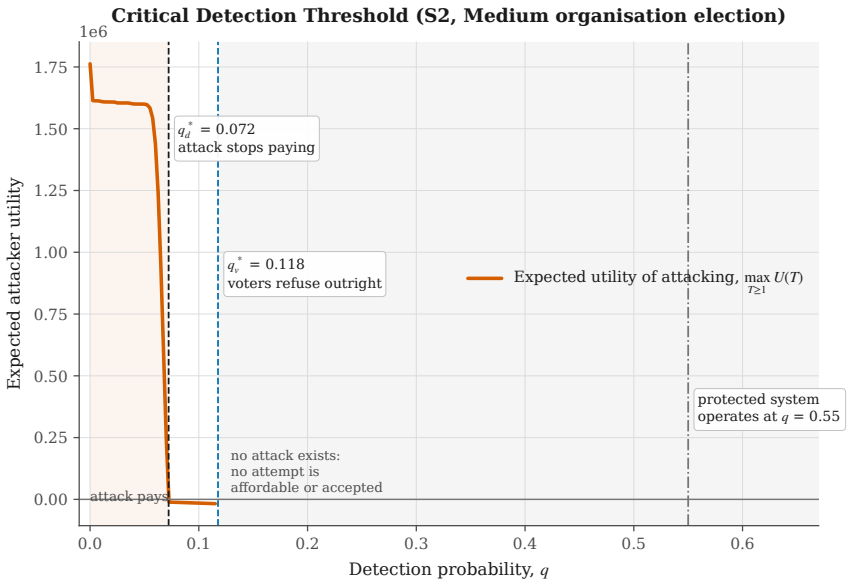


Figure 6. Expected utility of attacking as detection rises, for the medium-organisation scenario. Two thresholds are marked: q_d^*, where the attack stops paying, and q_v^*, where voters refuse outright.

The gap between the two thresholds is worth dwelling on. At q_d^* the attack is still perfectly possible — voters would accept, the budget suffices — but a rational attacker declines because the expected return has gone negative. At q_v^* the attack becomes impossible, because the voters themselves refuse. Deterrence arrives first and costs less; prevention arrives later and costs more. Table 7 places all five scenarios against both thresholds; every one of them sits far to the left of the protected configuration's operating point.

F5 — minimum viable defence (RQ4)

Making bribery unprofitable requires a detection probability of only 0.072 in the medium-organisation scenario and 0.050 at national scale — roughly one caught bribe in fourteen, and one in twenty. This is far below the 0.55 the modelled countermeasures deliver. The security-relevant conclusion is not that the countermeasures work, but that *much weaker ones would have*: the expensive part of the defence is buying certainty rather than deterrence.

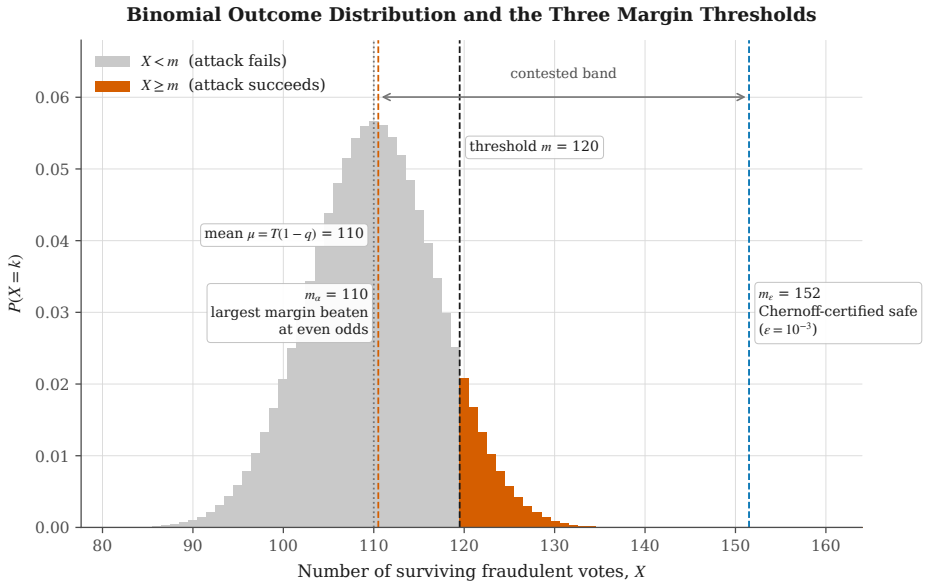
8.6.1 Break-even budget

The break-even budget answers the complementary question: how much would the attacker need before the attempt is worth making? Sweeping budget for the medium-organisation scenario gives $B^* = 249,480$ against a scenario budget of 250,000. Under the protected configuration the curve never crosses zero at any budget, because the bribe is not incentive-compatible at all.

8.6.2 The contested band of margins

Table 7 gives, for each scenario, the margin the attacker beats at even odds (m_α), the margin actually defended (m), and the margin the Chernoff bound certifies as safe (m_ε).

In four of the five scenarios the defended threshold sits inside its band or a little above the attacker-reachable margin; the exception is the low-budget scenario S5, where the defended margin exceeds m_α by a factor of about seven and the attack is out of reach by a wide margin rather than a narrow one. A certified margin m_ε exists for the national-scale scenario only — in the others the bound cannot certify even $m = n$ at $\varepsilon = 10^{-3}$, which the solver reports rather than silently omitting. At national scale the attacker reaches 19,000 votes at even odds while the bound certifies safety only from 19,515 — a contested band roughly 500 votes wide, with the defended threshold 19,015 sitting just inside its lower edge. That is the practical cost of the bound's conservatism: within that band it declines to certify margins that are in fact comfortably safe, and only exact computation resolves them. Figure 7 draws the same three thresholds on a single distribution.

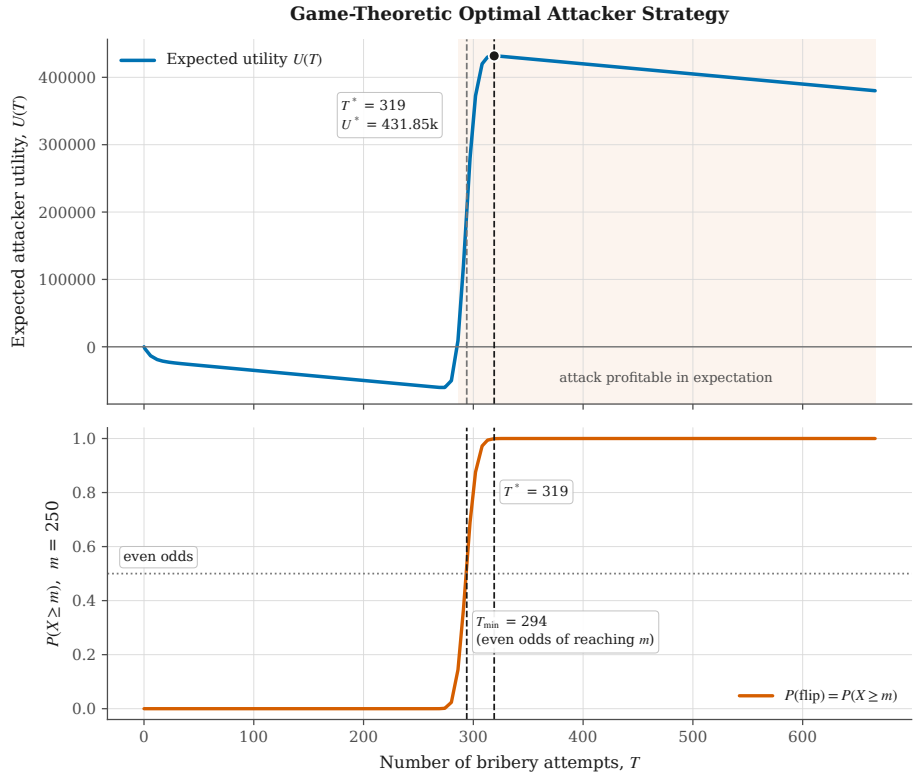


$X \sim \text{Binomial}(T = 200,\ 1 - q = 0.55)$; the shaded success region has probability $P(X \geq m) = 0.088$. Because X is integer-valued, region boundaries are drawn at half-integers (the m line sits at $m - 0.5$), so each line meets the edge of a bar rather than cutting through its centre. Margins at or below m_a are comfortably within reach; margins at or above m_ε are provably out of reach.

Figure 7. The binomial outcome distribution with its mean, the manipulation threshold, the attacker-reachable margin and the Chernoff-certified safe margin all marked.

8.6.3 The rational attacker does not spend everything

Figure 8 shows expected utility and flip probability against the number of bribery attempts. Utility is single-peaked: it rises while further bribes still move the flip probability, then declines once that probability saturates and each additional bribe buys only cost and detection risk.



Fixed scenario: $N = 2{,}000$, $m = 250$, $B = 100{,}000$, $R = 150$, $q = 0.15$. Utility peaks at $T^* = 319$ and then declines: once the flip probability saturates, each further bribe adds cost and detection risk but no additional chance of success. T^* is the best point on the evaluated grid rather than an exact argmax; the curve is flat near its peak, so nearby values of T are near-optimal too.

Figure 8. Expected attacker utility (top) and flip probability (bottom) against bribery attempts, with the utility-maximising T^* and the even-odds effort threshold $T_{\min}$ marked.

This matters for how the model is read. A budget-exhausting attacker — the implicit adversary of many security analyses — does strictly worse than a rational one, so optimising a defence against maximum spending optimises against the wrong opponent. The observation is the e-voting analogue of the selfish-mining result [8]: what an attacker *would choose* to do, not what they are capable of doing, is the security-relevant quantity.

8.7 F6 — Combining vectors defeats what no single vector can

8.7.1 No single vector suffices

Once each vector is held to a realistic capacity ceiling — forged registrations limited to 20% of the electorate by the registration window, injected ballots to 30% by the turnout anomaly they would otherwise create, in the sense that election forensics makes detectable [11] — something striking emerges from Table 8. Neither Sybil registration nor ballot stuffing has an *expected* yield that reaches the manipulation threshold in any scenario.

The distinction matters and was sharpened after review. The capacity ratio is $\mathbb{E}[X]/m$ — a mean, not a ceiling. A ratio below one means the attack falls short *on average*; success is still possible on an upward fluctuation. What makes the finding hold is how fast that possibility decays: at S1's bribery ratio of 0.97 the exact success probability is still 0.423, at S3's ballot-stuffing ratio of 0.99 it is 0.085, and by a ratio

of 0.74 it is numerically zero. So the accurate statement is that Sybil registration is effectively out of the running in every scenario, ballot stuffing retains a small residual chance in the two largest ones, and no single vector is a reliable route to the threshold — not that any of them is arithmetically impossible.

Table 8. Capacity of each vector at full effort, as a multiple of the manipulation threshold m (baseline configuration).

Scenario	Bribery	Sybil	Ballot Stuffing	Combined
S1	0.97	0.41	0.54	1.92
S2	1.00	0.74	0.97	2.71
S3	1.00	0.76	0.99	2.75
S4	1.00	0.15	0.20	1.35
S5	0.14	0.26	0.61	1.01

Note. Values are EXPECTED surviving votes over the threshold, so a value below 1.00 means the mean falls short — not that the threshold is unreachable. Success remains possible on an upward fluctuation, though its probability collapses quickly: at a ratio of 0.97 the exact $P(X \geq m)$ is still 0.42 for bribery in S1, while by a ratio of 0.74 it is numerically zero. Every entry here falls below 1.00; the budget-feasible optimal coalition clears it in four of the five scenarios (Table 9).

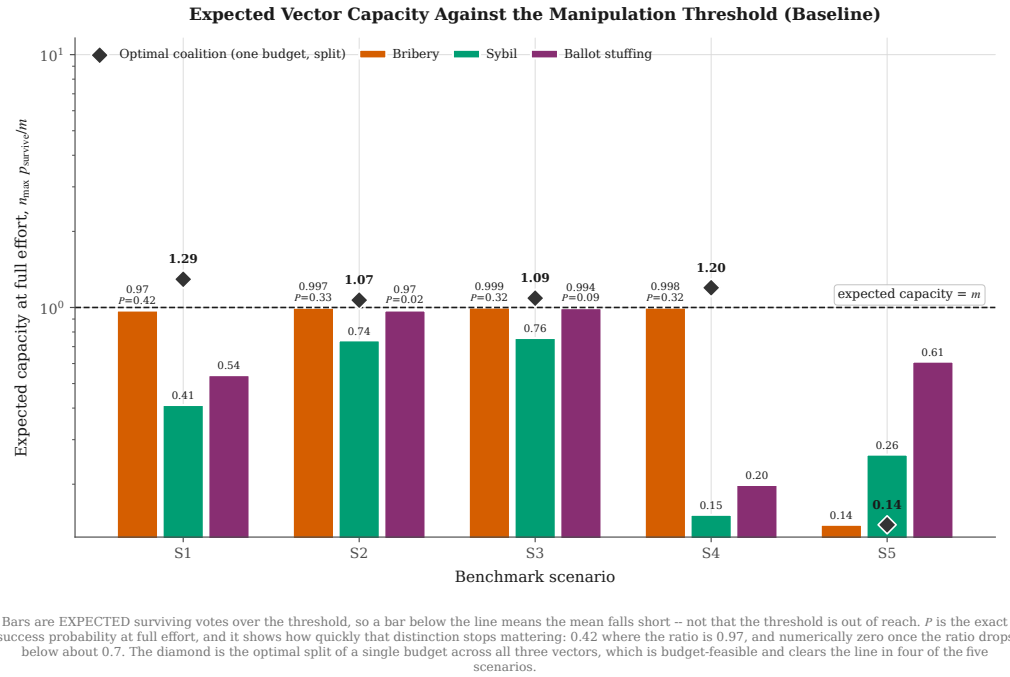


Figure 9. *Expected* capacity of each vector at full effort, as a multiple of the manipulation threshold. A bar below the line means the mean falls short, not that the threshold is unreachable — the annotated P is the exact success probability at full effort. The diamond is the optimal split of a single budget across all three vectors, which is budget-feasible.

Analysed one vector at a time, this reads as a reassuring result: no single vector is a dependable route to the threshold, and two of the three are hopeless. It is the wrong conclusion, and the reason it is wrong is that the attacker is not obliged to choose.

8.7.2 The mixed strategy

The search evaluates every way of splitting a single budget across the three vectors, at a 0.05 resolution over the budget simplex, whose corners are exactly the pure single-vector strategies. The optimum leaves the corners in four of the five scenarios: it is strictly interior in S1 $(0.35, 0.25, 0.40)$ and sits on an edge — two vectors funded, one not — in S2, S3 and S4. Only S5, where no allocation succeeds at all, returns a corner. The gain therefore comes from mixing *at least* two vectors rather than from using all three.

Table 9. Optimal multi-vector budget allocation per scenario (baseline configuration).

Scenario	Bribery	Sybil	Ballot	μ_{total}	Coalition P	Best pure	Pure P	Gain
S1	35%	25%	40%	45	0.999	Bribery	0.423	0.576
S2	0%	5%	95%	2,077	1.000	Bribery	0.328	0.672
S3	0%	5%	95%	20,700	1.000	Bribery	0.320	0.680
S4	10%	0%	90%	5,695	1.000	Bribery	0.316	0.684
S5	100%	0%	0%	76	0.000	Bribery	0.000	0.000

Note. Shares are the fractions of one budget allocated to each vector. Coalition success probability is computed exactly, by convolving the three binomial distributions rather than approximating their sum.

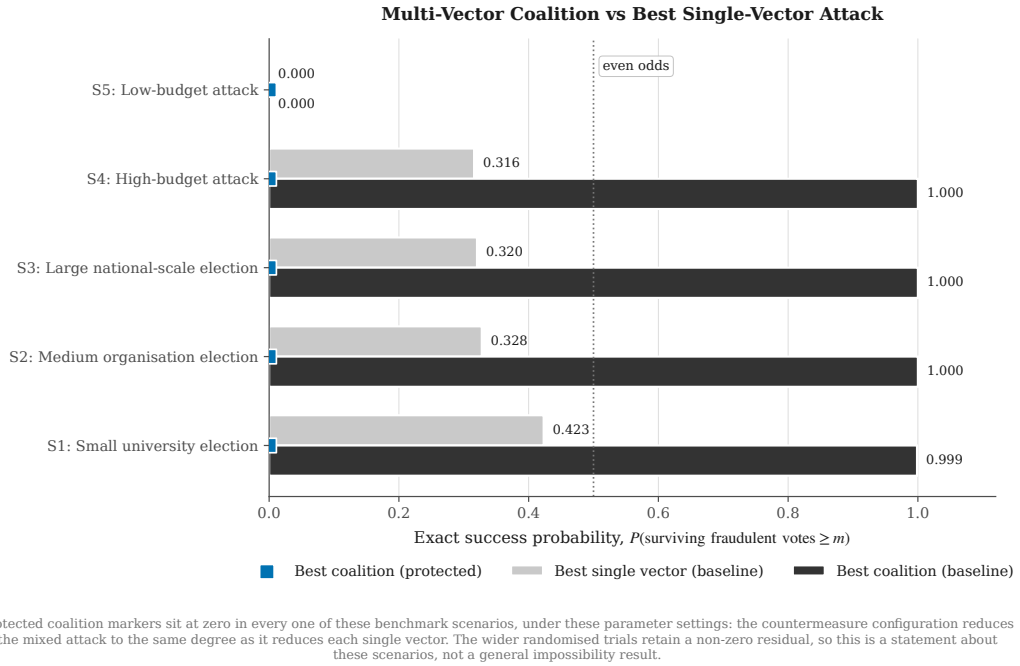


Figure 10. Best coalition attack against best single-vector attack per scenario, with protected coalition results overlaid.

F6 — multi-vector attacks (RQ5)

The optimal mixed strategy strictly outperforms the best single-vector strategy in 4 of 5 baseline scenarios, raising mean success probability from 0.277 to 0.800. In the high-budget scenario, no single vector exceeds a success probability of 0.316, yet the mixture reaches near-certainty. A security evaluation conducted one vector at a time therefore understates the achievable attack — not marginally, but qualitatively, by declaring infeasible an attack that succeeds.

The mechanism is capacity, not probability. Each vector saturates its own ceiling long before the budget is exhausted, leaving money that buys nothing more in that vector but a great deal in another. Diversification here is not about hedging risk; it is about escaping a binding constraint.

8.7.3 Robustness of the coalition finding

The coalition result depends on assumed relative costs and capacity ceilings, none of which is empirically grounded. The analysis therefore re-runs across 108 combinations of all four constants.

The coalition was at least as good as the best pure strategy in 100% of configurations and strictly better in 100%. The qualitative conclusion does not depend on the particular costs assumed — as it should not, since a mixed strategy can never be worse than the best pure one, the pure strategies being corners of the same simplex. What the sweep establishes is that the *strict* improvement is not an artefact of one convenient parameterisation.

8.7.4 Countermeasures against the coalition

Across the 5 benchmark scenarios, the protected configuration suppresses the coalition attack to the same degree as it suppresses each vector alone: mean coalition success probability under protection is 0.000. That figure is exactly zero for these particular benchmark scenarios under these parameter settings, and should be read as such rather than as a general claim that the mixed attack is impossible — the randomised trials, which sample far more widely, leave a residual success probability of 0.136 for Sybil attacks and 0.116 for ballot stuffing. The bribery gate closes in all 5 scenarios, since $0.55 > q_v^*$ everywhere, and the tightened acceptance and detection probabilities pull the other two vectors' capacity below the threshold. Coalition attacks are more dangerous than the single-vector analysis suggests, but they are not differently defended.

8.8 Scale effects

One further result bears on deployment, and it is not uniform across the vectors. Holding the manipulation threshold at a fixed 10% of the electorate and growing N, bribery success collapses from 1.000 to 0.000 and ballot stuffing from 0.237 to 0.000, because the attacker's budget ceiling $T = \lfloor B/R \rfloor$ does not scale with N while the threshold does.

Sybil registration behaves in the opposite direction, rising from 0.623 to 0.702, and the reason is structural: its attempt count is a fixed *fraction* of the electorate — forged registrations are capped at 20%

of N — so unlike a fixed budget it scales with the threshold it has to beat, and the binomial concentrates around a mean that stays a constant multiple of m. The budget argument simply does not apply to a vector whose capacity is denominated in voters rather than in currency. This is the scale-effect result stated precisely: growth protects against *budget-bound* vectors, not against *proportionally capped* ones.

The implication is not that large elections are safe. It is that large elections are safe *against a fixed budget*, and that the relevant attacker for a national election is correspondingly better resourced — which is exactly what scenario S3 models, with a budget of 3,000,000 and a resulting baseline flip probability of 0.320.

9 Discussion

9.1 What the ledger does and does not buy

The clearest result in this dissertation is also the least flattering to the technology it evaluates. Chain integrity held in every one of the 300 concrete elections, and in 44.0% of baseline bribery trials the attacker still changed the winner. Immutability and correctness are orthogonal properties. A ledger guarantees that what was recorded stays recorded; whether it should have been recorded is decided upstream, by identity verification, ballot validation and the incentives facing voters.

This is not an argument against blockchain voting so much as an argument about where its security budget belongs. The largest measured residual risk under protection is Sybil registration (0.136 mean success probability, roughly twice the protected bribery figure), which is exactly the vector the ledger does nothing about. Ballot stuffing is close behind at 0.116, so the residual is better described as concentrated in the two vectors that bypass the voter rather than in identity verification alone.

9.2 Deterrence is cheaper than prevention

The threshold analysis produces the most actionable finding. Making an attack unprofitable requires between four and eleven times less detection capability than the modelled countermeasures deliver: detection probabilities of 0.050 to 0.118 suffice across the 4 scenarios where detection is the binding constraint, against 0.55 provided. In the fifth scenario the attack is unprofitable at every detection level, so no threshold exists. If detection is expensive and roughly linear in cost — a reasonable assumption for audit sampling and identity checks — then most of the spend is buying certainty rather than deterrence.

Two caveats keep this from being a licence to under-invest. Deterrence assumes a rational attacker who computes expected utility and declines when it is negative; an attacker with non-monetary motives, or one who misjudges the detection rate, is not deterred by an unfavourable expectation. And q_d^* depends on the payoff V and penalty D, which are the least well-grounded parameters in the model. The qualitative conclusion — deterrence thresholds sit well below prevention thresholds — is robust; the specific numbers should be read as illustrative.

9.3 Single-vector analysis is not conservative

It is tempting to treat a per-vector evaluation as conservative, on the reasoning that each attack is assessed at full strength. The coalition results show this is backwards. Two of the three vectors have an expected yield below the threshold in every scenario, and combining them produced near-certain success in four of five. Single-vector analysis did find an attack: bribery was viable in 4 of the 5 scenarios, with success probabilities from 0.316 to 0.423. But it is the difference between a roughly one-in-three chance and a near-certainty that matters to a defender, and only the coalition analysis surfaces the latter.

The generalisable point is that capacity ceilings, not success probabilities, are what compose badly. Each ceiling is a hard limit within its own vector and irrelevant across vectors, so an attacker who spreads effort escapes all three at once. Any security argument resting on "the attacker cannot obtain enough of X" should be checked against an attacker who does not need all of it from X.

9.4 Answers to the research questions

RQ1

Success probability rises with budget and falls with detection, but the two are not symmetric. Budget effects are gradual and saturate at the capacity ceiling; detection effects are sharp, with success collapsing over a narrow band of q once the mean falls below the threshold. Detection is the more efficient lever.

RQ2

Monte Carlo reproduced the exact probability to a mean absolute error of 7.98e-5 for bribery (3.95e-4 and 3.94e-4 for the other two) and is the right tool where no closed form exists. The Chernoff bound never failed but was loose, by a mean of 0.0423 for bribery and 0.0260 and 0.0216 for the others; its value is that it *proves* rather than estimates.

RQ3

The protected configuration reduced success probability by 81.9%, 71.8% and 73.7% for the three vectors, and real winner-flip rates by up to 97.0%. The two largest residuals are Sybil (0.136) and ballot stuffing (0.116), both above the protected bribery figure of 0.064; the largest is Sybil, the vector the ledger does nothing about.

RQ4

Critical detection thresholds lie between 0.050 and 0.118 in the 4 of 5 scenarios where detection is the binding constraint — far below what the modelled countermeasures deliver. In the remaining scenario the attack never pays at any detection level.

RQ5

Yes. Mixed strategies strictly beat the best single vector in 4 of 5 scenarios, and the effect survives a 108-configuration robustness sweep.

10 Limitations and threats to validity

Independence. Attempts are modelled as independent Bernoulli trials with constant per-scenario probabilities. Real manipulation is correlated: a briber who finds one willing voter in a community finds more, and a detection mechanism that catches one forged registration often catches the batch. Positive correlation increases the variance of the outcome count without changing its mean, which fattens both tails — so the model likely understates both the best and worst cases. The direction of the effect on $\mathbb{P}(X \geq m)$ depends on which side of the mean the threshold sits, which is precisely the contested-band question of Section 8.6.

Homogeneous voters. All voters facing the same (q, R, P) reach the same decision, so the acceptance gate is binary at the population level. Real electorates are heterogeneous, and a distribution over reservation prices would replace the sharp gate with a smooth response curve. This is the single most consequential simplification in the bribery model and the most obvious extension.

Parameter grounding. The probabilities, the payoff V, the penalty D and the relative costs of the three vectors are plausible but not empirically calibrated. No public dataset gives the probability that a forged voter registration is detected. The sensitivity sweep addresses this for the coalition result; the threshold values should be read as illustrative of magnitude, not as measurements.

The blockchain is educational. The implementation mines real proof-of-work blocks and detects real tampering, but it is a single-process chain. It has no peer-to-peer network, no fork resolution, no economic security and no adversarial nodes. Consensus-layer attacks are consequently out of scope entirely.

Countermeasures as probabilities. Modelling identity verification as "catches a forgery with probability p_d" says nothing about how to achieve any particular p_d, and real mechanisms fail in correlated, adversarially-chosen ways rather than independently at a fixed rate.

Scope. Coercion, receipt-freeness, client-side malware, denial of service and consensus attacks are all excluded. A coerced voter differs from a bribed one in a way this model cannot express: the utility calculation of Equation (2) assumes the voter may decline.

11 Conclusion and future work

This dissertation modelled three e-voting manipulation vectors probabilistically and game-theoretically, evaluated them over 2,500 matched randomised trials and 300 genuinely mined elections, and inverted the model to find the parameter values at which each attack ceases to be viable.

The measured effect of the protected configuration is substantial — relative reductions of 81.9%, 71.8% and 73.7% in success probability, and up to 97.0% in real winner-flip rate. But the two findings that seem most worth carrying forward are the ones the aggregate numbers obscure. Attacks stop being worth mounting at detection probabilities between 0.050 and 0.118 in the 4 scenarios where detection binds — between four and eleven times below what a fully protected system provides — so deterrence is much cheaper than prevention. And an attacker who splits one budget across all three vectors beats the best

single-vector attacker in 4 of 5 scenarios, turning a roughly one-in-three chance into a near-certainty — so evaluating vectors independently is not the conservative choice it appears to be.

Underneath both sits the observation that motivated the work. Chain integrity was never violated in any trial, and elections were stolen anyway. The ledger faithfully recorded fraudulent votes as fraudulent votes, permanently and verifiably. Whatever blockchain-based voting is for, it is not for this.

Future work

Four extensions follow directly. Replacing the homogeneous voter with a distribution over reservation prices would turn the acceptance gate into a response curve and is the most natural next step. Introducing correlation between attempts, via a beta-binomial or an explicit contagion model, would test the independence assumption that the current tails depend on. Extending the coalition framework to a defender who allocates a detection budget across vectors would turn the analysis into a two-player game with a computable equilibrium, of which the present work computes only the attacker's best response. Finally, calibrating any of the probabilities against real electoral-fraud audit data would convert the illustrative threshold values into a usable defensive specification.

References

[1] B. Adida. Helios: Web-based Open-Audit Voting. In *Proceedings of the 17th USENIX Security Symposium*, pp. 335–348, 2008.

[2] G. S. Becker. Crime and Punishment: An Economic Approach. *Journal of Political Economy*, 76(2):169–217, 1968.

[3] J. Benaloh and D. Tuinstra. Receipt-Free Secret-Ballot Elections (Extended Abstract). In *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 544–553, 1994.

[4] M. Borge, E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly and B. Ford. Proof-of-Personhood: Redemocratizing Permissionless Cryptocurrencies. In *IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 23–26, 2017.

[5] H. Chernoff. A Measure of Asymptotic Efficiency for Tests of a Hypothesis Based on the Sum of Observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.

[6] P. Daian, T. Kell, I. Bentov and A. Juels. On-Chain Vote Buying and the Rise of Dark DAOs. *Hacking, Distributed* (IC3 technical report), July 2018. Available at https://hackingdistributed.com/2018/07/02/on-chain-vote-buying/. *Cited as a technical report; not peer reviewed.*

[7] J. R. Douceur. The Sybil Attack. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, LNCS 2429, pp. 251–260, 2002.

[8] I. Eyal and E. G. Sirer. Majority Is Not Enough: Bitcoin Mining Is Vulnerable. In *Financial Cryptography and Data Security (FC)*, LNCS 8437, pp. 436–454, 2014.

[9] P. Faliszewski, E. Hemaspaandra and L. A. Hemaspaandra. How Hard Is Bribery in Elections? *Journal of Artificial Intelligence Research*, 35:485–532, 2009.

[10] A. Juels, D. Catalano and M. Jakobsson. Coercion-Resistant Electronic Elections. In *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society (WPES)*, pp. 61–70, 2005.

[11] P. Klimek, Y. Yegorov, R. Hanel and S. Thurner. Statistical Detection of Systematic Election Irregularities. *Proceedings of the National Academy of Sciences*, 109(41):16469–16473, 2012.

[12] Z. Liu, N. C. Luong, W. Wang, D. Niyato, P. Wang, Y.-C. Liang and D. I. Kim. A Survey on Applications of Game Theory in Blockchain. *arXiv preprint* arXiv:1902.10865, 2019.

[13] P. McCorry, S. F. Shahandashti and F. Hao. A Smart Contract for Boardroom Voting with Maximum Voter Privacy. In *Financial Cryptography and Data Security (FC)*, LNCS 10322, pp. 357–375, 2017.

[14] M. Mitzenmacher and E. Upfal. *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Analysis*. 2nd ed., Cambridge University Press, 2017.

[15] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.

[16] H. O. Ohize, A. J. Onumanyi, B. U. Umar, L. A. Ajao, R. O. Isah, E. M. Dogo, B. K. Nuhu, O. M. Olaniyi, J. G. Ambafi, V. B. Sheidu and M. M. Ibrahim. Blockchain for Securing Electronic Voting Systems: A Survey of Architectures, Trends, Solutions, and Challenges. *Cluster Computing*, 28, 2025. DOI 10.1007/s10586-024-04709-8.

[17] S. Park, M. Specter, N. Narula and R. L. Rivest. Going from Bad to Worse: From Internet Voting to Blockchain Voting. *Journal of Cybersecurity*, 7(1):tyaa025, 2021.

[18] Rahul, P. Gulia and N. S. Gill. Articulation of Blockchain Enabled E-Voting Systems: A Systematic Literature Review. *Peer-to-Peer Networking and Applications*, 18:142, 2025. DOI 10.1007/s12083-025-01956-3.

[19] M. A. Specter, J. Koppel and D. Weitzner. The Ballot is Busted Before the Blockchain: A Security Analysis of Voatz, the First Internet Voting Application Used in U.S. Federal Elections. In *Proceedings of the 29th USENIX Security Symposium*, pp. 1535–1553, 2020.

[20] D. Springall, T. Finkenauer, Z. Durumeric, J. Kitcat, H. Hursti, M. MacAlpine and J. A. Halderman. Security Analysis of the Estonian Internet Voting System. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 703–715, 2014.

A Figure catalogue

The dissertation uses 10 figures. The set is deliberately small: an earlier draft generated thirty, of which several were variations on a single point, and the important figures were diluted by the rest. Each of the ten below carries a distinct step of the argument, and none can be removed without losing a claim the dissertation makes.

Figure numbers match the generated filenames — `fig_07_*` is Figure 7 — and `check_consistency.py` fails the build if the two ever drift apart. Every figure is produced by `generate_figures.py` from the experimental output at 320 DPI raster and vector PDF, and carries a machine-generated caption in `results/FIGURE_CAPTIONS.md`; none is drawn by hand.

Fig.	Subject	Carries
1	Method comparison: exact, Monte Carlo and Chernoff against detection probability	Cross-validation of the three methods; the detection collapse (RQ1, RQ2)
2	Rational voter utility surface and decision threshold	The consent gate that stops bribery independently of detection
3	Two-sided Chernoff certificate, both directions validated	That every scenario carries a proved guarantee (RQ2)
4	Three-attack comparison, baseline against protected	The headline countermeasure effect, and where residual risk sits (RQ3)
5	Election outcome flip rate on mined chains	That integrity and correctness are independent properties
6	Critical detection threshold q_d^*	How much detection is actually required (RQ4)
7	Binomial distribution with the three margin thresholds	What "attack success" means, and the contested band
8	Game-theoretic optimal attacker strategy	That a rational attacker stops short of spending everything (RQ1)
9	Vector capacity against the manipulation threshold	Why no single vector suffices on its own
10	Coalition against best single-vector attack	That mixing vectors beats committing to one (RQ5)

The analyses that earlier drafts illustrated with additional figures — the budget sweep, the feasibility surface, the electorate-size sweep, the break-even budget, the contested-margin band, the coalition simplex and the cost-sensitivity sweep — all still run, and their outputs remain in `results/csv/`. Only the figures were removed; no result was.

B Reproducing these results

1. `pip install -r requirements.txt`

2.	`python run_experiments.py -trials 2,500 -seed 42`	randomised paired trials
3.	`python run_election_attacks.py -trials 300 -seed 42`	concrete mined elections
4.	`python run_threshold_analysis.py`	thresholds and coalition
5.	`python generate_figures.py`	all 10 figures and captions
6.	`python generate_tables.py`	CSV and LaTeX tables
7.	`python generate_report.py`	the dissertation
8.	`python build_compendium.py`	the figures-and-results compendium
9.	`python build_submission.py`	this document (dissertation + appendix)
10.	`python check_consistency.py`	text-versus-data drift check
11.	`pytest -q`	122 tests

Every *experimental result* printed in this report is injected at build time from `results/json/` and `results/csv/` by `generate_report.py` rather than typed. The exceptions are deliberate and are not results: configuration values quoted from `config.py`, the worked example's arithmetic shown at reduced precision so that it reads on the page, and the analytic counter-examples of Section 3.7, which are computed and regression-tested in `tests/test_chernoff.py`. Re-running the pipeline with seed 42 reproduces this document exactly, and `check_consistency.py` fails the build if a stated count, figure number or macro drifts from what the project actually contains.